CRAFT: COMPOSITIONAL REWARD ALIGNMENT FOR TARGET-HIDDEN LOOP VERIFICATION

Anonymous authors
Paper under double-blind review

ABSTRACT

Valid loop invariants are closed under conjunction: useful clauses from different responses can prove a target even when no response does so alone. Yet standard training scores each response as an indivisible answer. We study this mismatch under *target-hidden loop verification*, where target assertions and postconditions Q are withheld during generation and training. CRAFT aligns both stages through rollout–decompose–pool–filter–compose. At inference, it decomposes responses into clauses, pools them across responses, filters the pool with syntax checking and Houdini, and composes the survivors into one invariant before restoring Q. Supervised fine-tuning (SFT) distills a filtered multi-response union into one response. Reinforcement learning (RL) filters the same pooled union once, assigns survivors back to their proposing rollouts, and combines their coverage reward with a bonus for rejections supplied by fewer peers. Its target-independent strength signal comes from reachable executions and conservatively filtered synthetic negatives. On 832 C programs, four GPT-5 Nano rollouts verify 49.0% of tasks, compared with 42.2% for the strongest external baseline in an end-to-end target-hidden evaluation with system-native fixed budgets. On the matched Qwen3-8B SFT path, RL leaves pass@1 nearly unchanged (41.9% to 40.8%) while increasing compose@4 from 75.1% to 79.4% and compose@32 from 79.0% to 82.7%.

1 INTRODUCTION

Loop invariants turn unbounded executions into finite proofs, but a formally valid invariant can still be useless. In this paper, *valid* specifically means inductive: the predicate is established when execution first reaches the loop and is preserved by every loop iteration. A verifier can check these obligations, yet its pass/fail outcome provides no graded measure of how much the invariant says: $true$ is valid for every loop but proves nothing about its behavior or desired outcome. A useful invariant must also be strong enough to establish downstream properties. Since the exact reachable-state set may be expensive to compute or inexpressible in the annotation language, generation must navigate many valid but weak over-approximations and receive credit for approaching stronger ones.

This search need not succeed in a single attempt. Validity is closed under conjunction: the conjunction of two valid invariants is valid and no weaker than either one. Useful clauses can therefore be accumulated across samples instead of demanding a complete answer from one response. Model responses expose such partial results as clauses: one may discover a range bound, another a relational equality, and together they may prove a target that neither proves alone. Clause-wise verification can discard invalid clauses while preserving valid ones, motivating the filtered clause pool as the inference object.

Response-level rewards are mismatched to this unit in two ways. First, verifiable rewards credited to whole responses chiefly sharpen the sampling distribution toward responses the policy already produces, improving pass@1 without widening what k samples can cover (Yue et al., 2025). For a clause pool, such sharpening toward a few individually complete responses is at best orthogonal and at worst collapses the diversity that composition exploits. Second, a complete response is also the wrong unit of training credit. A whole-response reward withholds credit from useful clauses whenever another clause in the same response fails. A reward computed independently for each response cannot distinguish a constraint that expands group coverage from one already repeated by every peer. Under repeated sampling, this can encourage the policy to reproduce easy high-reward

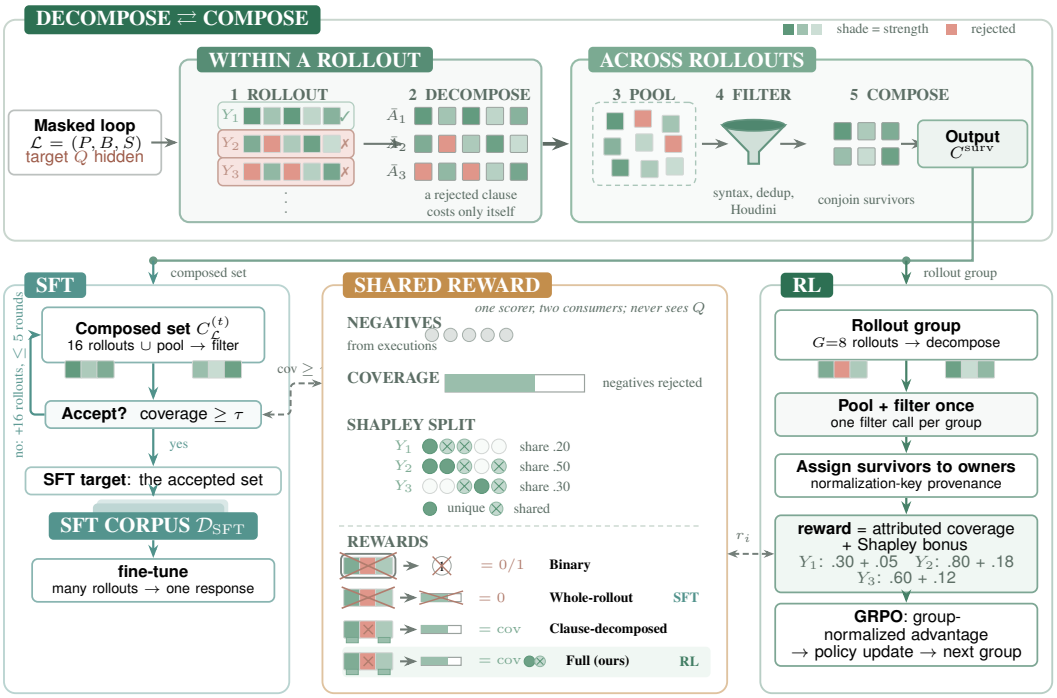

Figure 1: **Decompose–compose is the organizing principle.** Each target-hidden rollout is *decomposed* into clauses, and their union is *composed* into one filtered invariant (top). One target-independent reward (center) scores clause sets against execution-derived negatives and is shared by both training stages. *Left:* SFT accepts a composed set once its whole-rollout coverage of the negatives reaches τ, strengthening it with further rollout groups otherwise, and distills each accepted set into one response. *Center:* the reward combines coverage of the negatives with a closed-form Shapley bonus for shared rejections; the four credit rules ablated in Section 4.6 are drawn as pay/no-pay pictograms. *Right:* RL pools and filters each rollout group once, assigns survivors to owners by normalization-key provenance, combines attributed coverage and the Shapley bonus into one reward, and applies a GRPO update. The hidden target Q is restored only for the final proof of the composed output; neither generation nor training ever sees it.

clauses instead of exploring a clause pool whose members work together. Our central claim is that loop-invariant learning should optimize the verified composition obtained from a fixed rollout group, rather than treating every rollout as an independent final answer.

We study this claim under *target-hidden loop verification*. The model sees the function preconditions, executable prefix, loop guard, and body, but the target set Q is withheld until the generated invariant set has been fixed. The final test still uses the original task: after Q is restored, the verifier must establish the loop obligations and prove every target at its original program point.

Target hiding does more than remove a goal-copying shortcut: it changes the task from goal-directed proof completion to specification discovery. Conventional invariant generation assumes that the desired postcondition is given, providing a direct semantic direction for the search; in some benchmarks, the target itself, or a simple algebraic rearrangement of it, is already close to the required invariant. Under target hiding, the generator must instead infer behavioral facts solely from the initialization, guard, and state transitions. A verified inductive invariant serves both as a loop summary and, together with the exit condition, as a source of candidate postconditions. Our task can therefore be viewed as a form of verified specification discovery: the system first infers what the program guarantees and only afterward tests whether those inferred facts support the original verification goals. Because these summaries are not specialized to one visible assertion, they may also be reused across multiple downstream verification queries. Hiding Q additionally rules out target-specific proof failures and target success as training signals, making a graded, target-independent measure of invariant strength necessary.

CRAFT organizes inference around *rollout–decompose–pool–filter–compose*. Given a budget of k target-hidden responses, the pipeline extracts each invariant annotation as a clause, unions clauses across responses, removes syntax failures, and applies Houdini to retain a jointly valid subset. Only then is Q restored for the final proof. We measure this inference-time object with compose@k: the fraction of programs verified after composing the first k responses. Unlike pass@k, which requires at least one response to contain the complete proof, compose@k preserves useful partial answers. The objective is to make this curve both higher and faster to converge, so a small rollout budget recovers the compositions otherwise found only through much broader sampling.

Training aligns credit with the two levels of this inference procedure. *Within* each rollout, we project its extracted clauses onto a verifier-retained subset; one rejected clause does not erase credit earned by the others. We score the strength of that subset using the execution-derived negative examples it rejects, without adding a reward merely for well-formed syntax or whole-response validity. *Across* rollouts, we apply a closed-form Shapley allocation to a fixed coverage game induced by the pooled survivors and their rollout provenance: repeated rejections share the group-credit unit, whereas a rejection supplied by fewer peers receives a larger bonus. This allocation does not re-run filtering for subcoalitions and is not the Shapley value of the complete inference pipeline. Thus the reward of one rollout depends in part on what the other rollouts in its Group Relative Policy Optimization (GRPO) group discover (Shao et al., 2024). Synthetic supervised fine-tuning (SFT) additionally distills a filtered multi-response union into one individual response. Together, these stages make sampled responses more useful to the clause pool that the pipeline composes at inference.

Prior systems synthesize and prune candidate invariants at inference time, while learning-based methods train from verified outputs, solver feedback, or behavioral examples (Flanagan & Leino, 2001; Kamath et al., 2023; Cao et al., 2025; Si et al., 2020; Yan et al., 2025; Yang et al., 2026a; Huang et al., 2026). We instead train for the filtered clause pool used at inference: credit preserves useful clauses within a response and adds a group-conditioned bonus to the primary attributed-coverage reward.

Our experiments separate standalone success from composed inference. On 832 integer C programs, four target-hidden GPT-5 Nano responses with union and Houdini verify 408 programs (49.0%), compared with 351 (42.2%) for the strongest external baseline under each system's fixed budget. On the matched Qwen3-8B SFT path, the compositional reward keeps single-response accuracy nearly fixed (pass@1 41.9% to 40.8%) while shifting the composition curve upward (compose@4 75.1% to 79.4%; compose@32 79.0% to 82.7%).

Contributions. This paper makes three contributions:

- We formulate target-hidden loop verification around a fixed-budget rollout–decompose–pool–filter–compose objective. The evaluated object at inference is the verified composition measured by compose@k, not an isolated response.
- We align learning credit with that object at two levels, without using Q. Within a rollout, the unit of credit is the pooled-retained clause attributed by provenance, so an imperfect response is paid for what it contributes to the survivor pool; across rollouts, a closed-form Shapley allocation adds a bonus for non-redundant trace rejection. This allocation is exact only for the fixed coverage game after full-group filtering.
- We show that clause-level credit is what turns RL from sharpening into expanding compose@k, that group credit raises the curve further at every budget beyond one response, and that the trained policy reaches its untrained initialization's 32-response composition with four responses. On the SFT path, three training seeds give compose@4 79.4±0.5% and compose@32 82.7±0.5%. We audit the execution-derived examples that provide the graded strength signal.

2 Background and Related Work

Target-hidden loop verification. A task contains a loop $\mathcal{L} = (P, B, S)$ and targets $Q = \{(\ell, q_\ell)\}$, where P is the loop-entry predicate, B the guard, and S the body. In this paper, a *valid* invariant means an inductive predicate satisfying

$$P \Rightarrow I, \qquad \{I \wedge B\}\, S\, \{I\}.$$

For a finite clause set C, write $I_C = \bigwedge_{c \in C} c$, and call I Q-*sufficient* when it also proves every restored target: $\{I \wedge B\} S_{<\ell} \{q_\ell\}$ for an in-body assertion and $I \wedge \neg B \Rightarrow q_\ell$ after the loop. We denote the combined establishment, preservation, and location-specific obligations by $\mathrm{Verify}(\mathcal{L}, I, Q)$. During generation and training, CRAFT removes Q, contract comments, assertion calls, and error-label names; it restores the untouched source only after fixing I. Appendix A gives the supported scalar-integer, single-loop scope and a nonlinear example.

Invariant generation and behavioral strength. Classical methods combine abstract interpretation, templates, interpolation, recurrences, syntax-guided search, counterexamples, or dynamic traces (Cousot & Cousot, 1977; Karr, 1976; McMillan, 2003; Kincaid et al., 2017; Alur et al., 2013; Garg et al., 2014; Ernst et al., 2007). Modern neural systems generate, rank, filter, repair, or compose candidates with solvers (Si et al., 2020; Kamath et al., 2023; Chakraborty et al., 2023; Cao et al., 2025; Wen et al., 2024; Yang et al., 2026b). COINS and SpecRL show that execution-derived negatives can distinguish weak valid specifications from stronger ones (Yang et al., 2026a; Huang et al., 2026). CRAFT combines these ideas but makes the verifier-retained *cross-response clause union*, rather than one complete response, the learned object.

Set-level RLVR. With a whole-response verifiable reward, group-relative policy optimization can increase pass@1 by concentrating probability on already-supported answers without expanding large-k coverage (Yue et al., 2025; He et al., 2025). Pass@k objectives and rarity weighting instead reward a property of the sampled set (Walder & Karkhanis, 2025; Chen et al., 2025; He et al., 2025), but still treat each response as indivisible. CRAFT changes the credit unit: useful clauses survive even when sibling clauses fail, and a group bonus allocates the fixed union-coverage value toward rejections supplied by fewer peers.

3 METHOD

CRAFT uses the same rollout–decompose–pool–filter–compose abstraction for inference, SFT target construction, and RL credit. The target Q is absent from all three stages and is restored only for final evaluation.

3.1 POOL, FILTER, AND COMPOSE

For a masked loop $\mathcal{L}$, rollout i produces a response Y_i. Parsing and finite canonical normalization yield at most $K = 20$ clauses A_i. Given n responses, CRAFT forms

$$C_n^{\mathrm{pool}} = \bigcup_{i=1}^{n} \mathrm{set}(A_i), \qquad C_n = \mathsf{Filter}_{\mathcal{L}}(C_n^{\mathrm{pool}}), \qquad I_n = \bigwedge_{c \in C_n} c.$$

The filter removes parser failures and normalized duplicates, then applies Houdini until every survivor is jointly established and preserved (Flanagan & Leino, 2001). Frama-C/WP generates obligations; Alt-Ergo and Z3 must prove them within the resource limit, and unknown or timeout causes conservative deletion. Thus the implementation is sound relative to its accepted obligations but may remove a logically valid clause. Algorithm 1 records the shared computation.

Algorithm 1 Rollout–decompose–pool–filter–compose

Input: masked loop $\mathcal{L}$, clause sets $A_{1:n}$
Output: filtered clause set C

$C \leftarrow \mathrm{ConservativeDedup}(\bigcup_i A_i)$
remove parser-rejected clauses from C
repeat remove every $c \in C$ whose establishment or preservation is not proved under $\bigwedge C$
until no clause is removed; **return** C

The following standard facts justify composition; Appendix B gives proofs.

Proposition 1 (Conjunctive closure). *If $I_1, \ldots, I_m$ are valid invariants, then $\bigwedge_j I_j$ is valid and at least as strong as every conjunct.*

Proposition 2 (Mutual support). *A candidate set C is jointly inductive when $P \Rightarrow I_C$ and every $\{I_C \wedge B\}S\{c\}$, $c \in C$, holds; no clause need be preserved from itself alone.*

Theorem 3 (Candidate-relative Houdini optimality). *With exact logical decisions, Houdini returns the greatest jointly inductive subset of a finite candidate pool. Resource-bounded filtering retains no such completeness guarantee.*

3.2 Inference and Target-Independent Strength

At inference, CRAFT samples k masked responses, computes I_k, restores Q, and evaluates $\mathrm{Verify}(\mathcal{L}, I_k, Q)$. The population objective is

$$\max_{\pi} \ \mathbb{E}_{Y_{1:k} \sim \pi(\cdot|\mathcal{L})} \mathbf{1}[\mathrm{Verify}(\mathcal{L}, I_k, Q)],$$

estimated by compose@k. Unlike pass@k, it can succeed when no sampled response is sufficient alone.

Training cannot use Q, so CRAFT executes precondition-checked inputs to collect reachable loop-head states $\mathcal{E}$ and constructs a fixed budget of negative traces $\mathcal{N}$: relation-breaking perturbations, post-exit continuations, and random local fill. Cleaning removes witnessed reachable states, possible entries, duplicates, and unsupported state. A clause set rejects trace $\nu = \langle s_0, \ldots, s_T \rangle$ if some clause is false at some state,

$$\mathrm{rej}(C) = \{\nu \in \mathcal{N} : \exists c \in C, \exists t, s_t \not\models c\}, \qquad \mathrm{cov}(C) = \frac{|\,\mathrm{rej}(C)|}{|\mathcal{N}|}.$$

Coverage is therefore a trace-level empirical surrogate, not a semantic completeness measure. Appendix C.1 fixes construction and grouping.

3.3 SFT Distillation

For each masked training loop, the base model generates groups of 16 responses. CRAFT accumulates and filters their clause union until coverage reaches $\tau = 0.5$, for at most five rounds. Empty-negative programs are unscorable and excluded; unsuccessful searches are dropped. After removing duplicates, tautologies, and atomically implied clauses, the surviving union is serialized as one SFT target. Hence SFT distills multi-response search without exposing Q; exact acceptance and serialization appear in Appendix C.

3.4 Clause-Level Compositional RLVR

For one RLVR group $\mathcal{G} = \{1, \ldots, G\}$, CRAFT pools all raw clause sets, removes clauses contradicted by $\mathcal{E}$, and runs the same filter once. Using finite canonical keys, each survivor is assigned back to every rollout that proposed it. Let $C_i^{\mathrm{attr}}(\mathcal{G})$ be this attributed subset and $R_i(\mathcal{G}) = \mathrm{rej}(C_i^{\mathrm{attr}}(\mathcal{G}))$. The primary clause credit is

$$\mathrm{cov}_i(\mathcal{G}) = |R_i(\mathcal{G})|/|\mathcal{N}|.$$

This changes the rewarded event relative to ordinary RLVR: one failed sibling no longer erases credit for a useful retained clause. Group normalization alone cannot make that change; it only rescales the response-level rewards it is given.

To add a smaller complementarity incentive, fix the survivor sets obtained from the full group and define the coverage game $v_{\mathcal{G}}(\mathcal{S}) = |\bigcup_{i \in \mathcal{S}} R_i(\mathcal{G})|/|\mathcal{N}|$. If $f_{\mathcal{G}}(\nu) = \sum_j \mathbf{1}[\nu \in R_j(\mathcal{G})]$, its Shapley allocation has the closed form

$$\phi_i(\mathcal{G}) = \frac{1}{|\mathcal{N}|} \sum_{\nu \in R_i(\mathcal{G})} \frac{1}{f_{\mathcal{G}}(\nu)}, \qquad \sum_i \phi_i(\mathcal{G}) = v_{\mathcal{G}}(\mathcal{G}). \tag{1}$$

Thus each union-covered trace contributes exactly one unit of group credit, shared inversely with redundancy. This is exact for the fixed coverage game; it does not re-filter coalitions and is not a Shapley value of the complete proof pipeline.

The generation reward is

$$r_i(\mathcal{G}) = 1.0\ \mathrm{cov}_i(\mathcal{G}) + 0.3\,\phi_i(\mathcal{G}). \tag{2}$$

GRPO normalizes r_i within groups of eight and applies the token-level policy update specified in Appendix C.2. Ordinary coverage remains primary; the Shapley term is a complementarity bonus. Support-only clauses that reject no negative receive no direct reward even if they enable another clause's inductiveness, which bounds the surrogate's scope rather than its verifier-checked final correctness.

4 Experiments

We ask how composition scales with sampling (RQ1), how much each pipeline component contributes (RQ2), how CRAFT compares with specialized tools (RQ3), how much RL adds (RQ4), and what drives the training gain (RQ5).

4.1 Experimental Setup

Data and judge. Training uses 4,992 RL and 33,346 SFT records from 31,865 target-masked single-loop programs; SFT targets are generated with GPT-5.6 Luna and verified by Frama-C/WP (Appendix G). Evaluation covers 832 C programs: 316 linear and 50 NLA tasks from Code2Inv, SyGuS, SV-COMP, and LIPuS, plus 466 Loopy tasks (Si et al., 2020; Alur et al., 2013; Beyer, 2023; Yu et al., 2023; Kamath et al., 2023). No train–test match remains under exact, alpha-renamed, or constant-abstracted comparison (Appendix F). Success requires Frama-C/WP to prove induction and every restored target with 5 s per obligation; all other outcomes are failures.

Inference protocol. CRAFT runs share masking, prompting, clause processing, and Houdini; external tools retain their native target-hidden orchestration. Decoding uses temperature 1, top-$p = 1$, no re-roll, and no target feedback. API models disable reasoning and cap completions at 8,192 tokens; local checkpoints use non-thinking decoding and 2,048 emitted tokens. RQ1 reuses one 32-response pool per local checkpoint and task. Appendix H fixes all serving and training settings.

Provisional presentation values. Red values and † rows are internally consistent mock results, not empirical findings; all must be replaced before submission.

Metrics. pass@k measures whether any response proves Q, using the unbiased estimator $1 - \binom{n-s}{k}/\binom{n}{k}$ from s successes in n saved responses. compose@k instead unions the first k responses, filters them, and reports the verified-program fraction; it is a fixed-prefix metric, not an average over subsets. We also report k_{95}, the smallest $k \in \{1, 4, 8, 16, 32\}$ reaching 95% of compose@32.

4.2 RQ1: How Do pass@k and compose@k Scale?

We sample 32 target-hidden responses per task from six base checkpoints: Qwen3-1.7B, Qwen3-4B, Qwen3-8B, Qwen3-14B, Qwen3-30B-A3B (Yang et al., 2025), and Llama 3.1 8B. Complete curves, k_{95}, and per-stratum breakdowns appear in Table 10.

At k=1, compose@1 exceeds the unbiased pass@1 estimate by 14.9–42.4 points across checkpoints. In the retained response pools, compose@16 is within 2.1 points of compose@32 on every checkpoint ($k_{95} \in \{8, 16, 32\}$; Table 10). In contrast, pass@8 reaches only 72–74% of pass@32 on the five Qwen checkpoints and gains another 5.0–7.8 points by k=32. Llama 3.1 8B is harder in standalone proof generation—pass@32 is 13.8%—but shows the same compositional pattern: compose rises from 29.2% at k=1 to 56.0% at k=16, then changes by only 0.1 points.

Insight. Composition finds most of its useful clauses within the first dozen samples, whereas standalone proofs require wider sampling. The compose@32 values (41.4–67.8%) further suggest that these pools remain limited by clause quality or diversity: extending the budget to 32 responses does not remove the gap.

4.3 RQ2: What Does Each Component Contribute?

The main neural experiment evaluates saved responses per program at budgets $k{=}1$ and $k{=}8$ and decomposes the pipeline into cumulative stages. Framework-free rows judge each response as one indivisible conjunction (pass@k); pipeline rows decompose the same responses into clauses, filter them, and compose the survivors (compose@k). Zero denotes the untrained checkpoint. The three Qwen3 models add the training stages (+SFT, +SFT+RL) and two ablation rows that repeat the trained checkpoints without the pipeline; Llama 3.1 8B reports the SFT rows without RL.

	Zero		SFT		SFT+RL	
Backbone	pass	compose	pass	compose	pass	compose
Qwen3-4B	14.1	51.2	60.0	76.2	57.2	80.0
Qwen3-8B	20.5	59.0	61.1	77.5	60.3	81.6
Qwen3-14B	21.3	65.0	63.5	78.4	62.9	81.5
Llama 3.1 8B	5.4	54.6	55.8	72.5	–	–

Table 1: Aggregate component comparison at $k{=}8$ (All / 832, %). Pass treats each response as one conjunction; compose uses clause decomposition, Houdini filtering, and union. Red values are mock results. Per-stratum $k{=}1, 8$ results are in Table 12.

Composing lifts every model by 28.4–42.4 points at $k{=}1$ and by 37.1–49.2 points at $k{=}8$ in aggregate. Per-model probe curves appear in Table 10.

Insight. Across the three Qwen backbones, SFT raises compose@8 from 51.2–65.0% to 76.2–78.4%. The matched RL rows then raise compose@8 to 80.0–81.6%, while their framework-free pass@8 changes from 60.0–63.5% to 57.2–62.9%. The same decomposition transfers to Llama 3.1 8B: SFT raises compose@8 from 54.6% to 72.5%, compared with a framework-free pass@8 of 55.8%. Thus SFT improves the quality of individual responses, whereas the RL gain is concentrated in the filtered union evaluated by compose@k.

4.4 RQ3: How Does CRAFT Compare with Specialized Tools?

We compare CRAFT with the target-hidden pipelines of AutoSpec, SESpec, Clause2Inv, Loopy, and Daikon (Wen et al., 2024; Yang et al., 2026b; Cao et al., 2025; Kamath et al., 2023; Ernst et al., 2007). All LLM-based systems use GPT-5 Nano, and all outputs are judged on the same untouched programs by Frama-C/WP. Each system retains its native, predeclared orchestration, so model-call and token budgets differ; Table 2 reports both token and runtime costs. Unsupported inputs count as failures in the common 832-program denominator.

The Naive baseline uses one call without invariant-specific instructions. In contrast, CRAFT prompts for a diverse clause pool and filters the result before composition. Consequently, its raw responses have lower standalone validity than Naive (pass@1: 3.61% vs. 16.47%), while its filtered composition reaches 33.89% at the same one-response budget.

System	Linear / 316	NLA / 50	Loopy / 466	All / 832	Tokens / task	Time / task
AutoSpec	47.78%	6.00%	42.27%	42.19%	2,181.72	27.34 s
SESpec	28.80%	4.00%	33.05%	29.69%	22,081.61	68.50 s
Clause2Inv	20.89%	0.00%	0.00%	7.93%	1,056.25	8.41 s
Daikon	23.10%	0.00%	21.67%	20.91%	0	4.89 s
Loopy	20.25%	0.00%	19.74%	18.75%	14,513.37	147.16 s
Naive	15.19%	2.00%	18.88%	16.47%	225.95	2.98 s
CRAFT (pass@1)	2.53%	0.00%	4.72%	3.61%	1,135.83	7.86 s
CRAFT (compose@1)	34.81%	22.00%	34.55%	33.89%	1,136.82	24.52 s
CRAFT (compose@4)	48.10%	30.00%	51.72%	49.04%	1,905.10	46.40 s
CRAFT (compose@8)	55.38%	44.00%	56.87%	55.53%	2,929.47	69.99 s

Table 2: Common target-hidden comparison with reasoning_effort=none, using the same untouched inputs and Frama-C/WP final judge. All LLM-driven rows share the same fixed backbone model; tokens and time are mean totals per task. The three compose rows are recomputed from one archived ten-rollout pool by prefix-subset composition, unioning the first k responses through the same filter chain; their tokens count one prompt plus k archived completions, and their time adds the measured filter-and-judge time at budget k to the single-request latency. The Loopy column is the benchmark stratum, whereas the Loopy row is the tool of Kamath et al. (2023).

The archived reasoning-enabled comparison appears in Table 20. On three further API models the same pairing lifts compose@1 over pass@1 by 18.7–23.4 points (mean 20.9; Table 19), compared with the backbone's 30.3: the gain appears across models and compresses as single-shot accuracy rises. Under this setting, increasing CRAFT from four to eight candidates raises verification from 49.0% to 55.5%.

Appendix I.8 documents system-specific interface constraints under target hiding. The complete per-tool results and the archived medium-reasoning comparison also appear there.

Insight. Under native fixed orchestration, four-response CRAFT verifies 49.0% of all tasks versus 42.2% for AutoSpec, while eight responses reach 55.5%. Because calls and runtimes differ, this establishes end-to-end utility rather than strict equal-compute dominance; the matched Naive comparison isolates the 17.4-point gain from one-response filtering and composition.

4.5 RQ4: How Much Does Reinforcement Learning Improve Inference?

We evaluate matched Qwen3-8B before–after RL paths from the Zero and SFT initializations: Zero is the untrained checkpoint and RL-Zero its RL result; SFT and SFT+RL are the corresponding SFT-path checkpoints. Inference is fixed within each pair.

What to expect. If RL with whole-response credit merely sharpens the distribution (Yue et al., 2025), it should raise pass@1 while leaving or degrading large-k coverage. Clause-level credit with group allocation predicts the opposite pattern: pass@k stays near the initialization, while compose@k shifts up at small budgets without degrading at large ones. We write k^* for the smallest budget in the measured grid at which the RL policy's compose@k reaches its initialization's compose@32. The SFT stage distills the filtered multi-response union into single responses (Section 3.3); the matched SFT-path row appears in Table 3, with full curves across the backbones in Appendix I.2.

Checkpoint	Stage	pass@1	compose@1	compose@8	compose@32	k_{95}
Zero	before RL	8.3%	43.8%	59.0%	62.5%	16
RL-Zero	after RL	6.8%	57.9%	71.3%	74.3%	8
SFT	before RL	41.9%	65.5%	77.5%	79.0%	4
SFT+RL†	after RL	40.8%	72.2%	81.6%	82.7%	4

Table 3: Matched Qwen3-8B evaluation before and after RL. The † row is the mock mean of three SFT+RL seeds; seed-level values appear in Table 16.

Insight. Response-level accuracy does not improve after RL (Figure 3, left; pass@1 40.8% vs. 41.9%). The redistribution is instead steered toward the complementary tail of the clause pool: compose@k^* with k^*=4 matches its initialization's compose@32 (79.0%), an effective eight-fold budget reduction, and compose@32 itself does not degrade (82.7% vs. 79.0%). RL thus packs into k^* samples the useful clauses its initialization produces only across thirty-two. In the accompanying clause-frequency diagnostic, 31% of RL-only retained normalization keys appear at most once in 32 matched initialization responses (empirical frequency at most 3.1%).

4.6 RQ5: What Drives the Training Gain?

We run two ablations, one over the reward function and one over the negative sampler. Both train Qwen3-8B from the Zero initialization on the curated pool; runs within an ablation differ only in the reward or the sampler. Appendix I.4 gives the exact reward definitions and complete numbers.

Reward-function ablation. Four variants form a ladder. Binary pays $\{0, 1\}$ when a nonempty response has every admitted clause retained by the pooled group filter; Whole-rollout pays $\mathrm{cov}_i(\mathcal{G})$ under the same condition; Clause-decomposed pays it for the surviving subset $C_i^{\mathrm{attr}}(\mathcal{G})$; Full adds the group (Shapley) credit $w_g\phi_i(\mathcal{G})$ of Equation (2). The first two are response-level credit, the last two clause-level; all share the clause cap K=20.

Insight. The reward unit determines whether RLVR sharpens responses or expands the composable pool. Whole-rollout credit raises pass@1 from 8.3% to 25.4% but lowers compose@32 from 62.5% to 33.2%; binary credit reaches only 27.9%. Paying retained clauses instead raises compose@32 to 70.2%, and the fixed-game group bonus adds 4.1 points to 74.3%. Thus most of the gain comes from removing the all-or-nothing response gate; inverse-frequency group allocation supplies a smaller, consistent complementarity gain.

Negative-sampler ablation. Under the full reward, the structured sampler (relation and post-exit traces followed by random budget fill, Section 3.2) exceeds uniform random perturbations by 5.5–6.7 compose@k points at every budget, whereas pass@k differs by -1.8 to $+1.6$ points (Table 18, Appendix I.4).

Does negative coverage predict hidden-target utility? We score 6,190 saved target-hidden candidate sets from eight generation pipelines on the structured negatives, then use the restored-target Frama-C/WP verdict only as an evaluation label. Among the 6,182 candidates with a continuous coverage score, coverage predicts target success with AUROC (area under the ROC curve) 0.793 (program-clustered 95% CI 0.775–0.811) and AUPRC (area under the precision–recall curve) 0.768, against a 0.525 positive-rate baseline. More stringently, ranking candidates only against other candidates for the same program gives macro AUROC 0.802 (0.783–0.820); a within-program permutation test gives $p < 2 \times 10^{-4}$. The association is strong on Linear and Loopy but weak on NLA, exposing a limitation of the current negative construction. Appendix I.5 gives the protocol, strata, and diagnostic curves.

5 Conclusion and Limitations

CRAFT trains for the filtered clause composition used at inference. SFT distills a multi-response union, while target-independent RL combines primary attributed coverage with a smaller complementarity bonus. On the matched Qwen3-8B SFT path, RL preserves pass@1 (41.9% versus 40.8%) while raising compose@4 from 75.1% to 79.4% and compose@32 from 79.0% to 82.7%. Ablations attribute the larger effect to clause-level credit and the additional gain to group allocation.

Evidence is limited to scalar-integer single-loop C, Frama-C/WP, and the tested model families; fixed-group provenance is not full coalition Shapley. The approach may transfer when outputs decompose into mechanically checkable units and execution-derived negatives distinguish weak from strong solutions. Appendix J details these boundaries.

AI Use Statement

Generative AI tools are integral to the evaluated system: they produce the invariant candidates and synthetic SFT data described in the paper. They were also used during research to refine hypotheses, methodology, and experiments; formulate and check mathematical claims; implement and test software; analyze and interpret experimental artifacts; prepare figures and tables; search and summarize literature; and draft, edit, and translate manuscript text. The authors reviewed all AI-assisted work and tested generated code. They take responsibility for independently validating the final content, empirical claims, and released artifacts.

Reproducibility Statement

The method and objective are specified in Section 3; Appendices C–H give the implementation defaults, prompt, training interface, data-overlap audit, and evaluation protocol. Appendix I organizes the available measurements, including per-seed results.

References

Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *Proc. Formal Methods in Computer-Aided Design (FMCAD)*, pp. 1–8, 2013.

Dirk Beyer. Competition on software verification (SV-COMP). `https://sv-comp.sosy-lab.org/`, 2023.

Weining Cao, Guangyuan Wu, Tangzhi Xu, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. Clause2Inv: A generate-combine-check framework for loop invariant inference. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):1009–1030, 2025. doi: 10.1145/3728920.

Saikat Chakraborty, Shuvendu K. Lahiri, Sarah Fakhoury, Madanlal Musuvathi, Akash Lal, Aseem Rastogi, Aditya Senthilnathan, Rahul Sharma, and Nikhil Swamy. Ranking LLM-generated loop invariants for program verification. In *Findings of the Association for Computational Linguistics: EMNLP*, pp. 9164–9175, 2023. doi: 10.18653/v1/2023.findings-emnlp.614.

Zhipeng Chen, Xiaobo Qin, Youbin Wu, Yue Ling, Qinghao Ye, Wayne Xin Zhao, and Guang Shi. Pass@k training for adaptively balancing exploration and exploitation of large reasoning models. *arXiv preprint arXiv:2508.10751*, 2025.

Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proc. 4th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pp. 238–252, 1977.

Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69:35–45, 2007.

Cormac Flanagan and K. Rustan M. Leino. Houdini, an annotation assistant for ESC/Java. In *Proc. Int. Symp. of Formal Methods Europe (FME)*, pp. 500–517. Springer, 2001.

Pranav Garg, Christof Löding, P. Madhusudan, and Daniel Neider. ICE: A robust framework for learning invariants. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 69–87. Springer, 2014.

Andre He, Daniel Fried, and Sean Welleck. Rewarding the unlikely: Lifting GRPO beyond distribution sharpening. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025.

Zhechong Huang, Zhao Zhang, Zeyu Sun, Huifeng Sun, and Yingfei Xiong. Reinforcement learning with negative tests as completeness signal for formal specification synthesis. *arXiv preprint arXiv:2604.05820*, 2026.

Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K. Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. Finding inductive loop invariants using large language models. *arXiv preprint arXiv:2311.07948*, 2023.

Michael Karr. Affine relationships among variables of a program. *Acta Informatica*, 6(2):133–151, 1976.

Zachary Kincaid, Jason Breck, Ashkan Forouhi Boroujeni, and Thomas Reps. Compositional recurrence analysis revisited. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 248–262, 2017.

Chang Liu, Xiwei Wu, Yuan Feng, Qinxiang Cao, and Junchi Yan. Towards general loop invariant generation: A benchmark of programs with memory manipulation. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Kenneth L. McMillan. Interpolation and SAT-based model checking. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 1–13. Springer, 2003.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Xujie Si, Aaditya Naik, Hanjun Dai, Mayur Naik, and Le Song. Code2Inv: A deep learning framework for program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 151–164. Springer, 2020.

Christian Walder and Deep Karkhanis. Pass@k policy optimization: Solving harder reinforcement learning problems. In *Advances in Neural Information Processing Systems*, volume 38, 2025.

Cheng Wen, Jialun Cao, Jie Su, Zhiwu Xu, Shengchao Qin, Mengda He, Haokun Li, Shing-Chi Cheung, and Cong Tian. Enchanting program specification synthesis by large language models using static analysis and program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pp. 302–328. Springer, 2024.

Chuanhao Yan, Fengdi Che, Xuhan Huang, Xu Xu, Xin Li, Yizhi Li, Xingwei Qu, Jingzhe Shi, Chenghua Lin, Yaodong Yang, Binhang Yuan, Hang Zhao, Yu Qiao, Bowen Zhou, and Jie Fu. Re:Form—reducing human priors in scalable formal software verification with RL in LLMs: A preliminary study on Dafny. *arXiv preprint arXiv:2507.16331*, 2025.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Fanpeng Yang, Xing Li, Shuling Wang, Jie An, Zeyu Sun, Shenghua Feng, Wenhan Wang, Weiyi Wang, Naijun Zhan, and Fanjiang Xu. How powerful are LLMs in generating formal program specifications? In *Proc. Int. Conf. on Machine Learning (ICML)*, 2026a.

Fanpeng Yang, Xu Ma, Shuling Wang, Xiong Xu, Qinxiang Cao, Naijun Zhan, Xiaofeng Li, and Bin Gu. Integrating symbolic execution with LLMs for automated generation of program specifications. *arXiv preprint arXiv:2506.09550*, 2026b.

Shiwen Yu, Ting Wang, and Ji Wang. Loop invariant inference through SMT solving enhanced reinforcement learning. In *Proc. ACM SIGSOFT Int. Symp. on Software Testing and Analysis (ISSTA)*, pp. 175–187, 2023. doi: 10.1145/3597926.3598047.

Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems*, volume 38, 2025.

Appendix guide. The appendices separate method justification, reproducibility material, data audits, experimental results, and reward-path diagnostics. The table below states the question answered by each appendix.

	Topic	Purpose
A	Scope and example	Defines the supported program class and shows why hiding the target prevents copying.
B	Formal proofs	Proves conjunction, mutual support, Houdini greatestness, and the closed-form coverage-game Shapley value.
C	Implementation	Records sampler, verifier, reward, and GRPO details needed to reproduce the method.
D	Generation prompt	Gives the exact target-hidden model instruction and output grammar.
E	Training interface	Documents the reward-service API and selectable ablations.
F	Overlap audit	Tests whether training programs duplicate or structurally overlap evaluation programs.
G	Training-pool curation	Describes how SFT and RL training pools were constructed and filtered.
H	Experimental protocol	Fixes evaluation, decoding, verification, and training configurations.
I	Complete results	Reports full probe, training, ablation, sampler-validity, cross-model, and tool results.
J	Limitations	Delimits program scope, serving, verifier, candidate-pool, and feedback limitations.
K	No-thinking setting	Explains why local-model generation disables chain-of-thought consistently.
L	Pooled reward path	Shows why clauses must be pooled before filtering, defines attribution and reward, and measures runtime only.

A Scope and Target-Hiding Example

We focus on numerical loops because their arithmetic equalities and inequalities, including polynomial relations, admit scalable automatic checking. This controlled setting factors out arrays, heaps, recursive data structures, auxiliary functions, and inductive predicates, allowing the study to isolate algebraic discovery and loop dynamics. Concretely, we consider one braced while loop over scalar C integers, including nonlinear updates. Even this restricted setting can hide nonlinear relations across coupled recurrences. For example:

```c
void f(int k, int z) {
  int x = 1, y = 1, c = 1;
  while (c < k) { c++; x = x*z + 1; y = y*z; }
  /*@ assert 1+x*z-x-z*y == 0; */
}
```

Case study: target visibility turns discovery into copying. If the assertion is visible to the generator, the task is nearly trivial: the assertion itself can be copied verbatim as a loop invariant. It holds initially because $x = y = 1$. Writing $h(x, y; z) = 1 + xz - x - zy$, where the semicolon separates the loop-constant parameter z, one loop iteration gives

$$h(xz + 1, yz; z) = 1 + (xz + 1)z - (xz + 1) - z(yz) = z\,h(x, y; z),$$

so the predicate $h(x, y; z) = 0$ is inductive and directly discharges the exit assertion. Under the target-hidden protocol, however, the model sees the initialization and the two coupled recurrences but not the assertion, and must independently recover this relation. The archived non-reasoning rollout pools do so in three algebraically equivalent forms:

Model	First verified prefix	Rediscovered invariant clause
GPT-5	compose@1	$x - 1 = z(x - y)$
DeepSeek V4-Flash	compose@4	$x(z - 1) = zy - 1$
Claude Sonnet-4.6	compose@8	$x(z - 1) = yz - 1$

All three clauses rearrange to the hidden assertion. None of the ten raw responses in any of these three pools passes as a whole on this program; the inference filter removes incompatible siblings and retains the useful relation. Thus target hiding prevents a copy-the-goal shortcut, while the successful compositions demonstrate recovery from loop dynamics rather than access to the postcondition.

B Proofs of Compositionality

This section contains all proofs for the formal composition claims in Section 3.1. Write $\mathcal{L} = (P, B, S)$, where P is the loop-entry predicate, B the guard, and S the body transition.

B.1 Conjunction of Valid Invariants

Proof of Proposition 1. For each j, validity gives $P \Rightarrow I_j$; hence $P \Rightarrow \bigwedge_j I_j = I$. Suppose $I \wedge B$ holds before one execution of S. Then $I_j \wedge B$ holds for every j, so preservation of I_j establishes every I_j afterward. Therefore I is preserved and is a valid invariant.

Because a conjunction implies each conjunct, $I \preceq I_j$. It satisfies $I \prec I_j$ precisely when the reverse implication fails. Since

$$I_j \Rightarrow I \quad \Longleftrightarrow \quad I_j \Rightarrow \bigwedge_{i \neq j} I_i,$$

the stated condition follows. In particular, two valid invariants form a strict strengthening of both exactly when neither invariant implies the other; without this non-redundancy condition, conjunction is only guaranteed to be no weaker. □

B.2 Mutual Support Among Arbitrarily Many Clauses

Proof of Proposition 2. The first premise is exactly establishment of I_C. For preservation, assume $I_C \wedge B$ before executing S. The second premise establishes each c_j afterward. Since every conjunct then holds in the same successor state, their conjunction I_C also holds. Thus $\{I_C \wedge B\} S \{I_C\}$, and I_C is valid.

No standalone preservation premise appears in this argument. A clause may use all other conjuncts in the pre-state, so clauses that fail preservation separately may still be preserved jointly. Establishment is different: $P \Rightarrow I_C$ implies $P \Rightarrow c_j$ for every j. Hence a candidate false at some loop-entry state cannot be rescued by conjunction. □

The phenomenon is not limited to pairs. For any $m \geq 2$, consider the cyclic transition, where x_i' denotes the value of x_i after one execution of S,

$$x_1' = x_2, \quad x_2' = x_3, \quad \ldots, \quad x_{m-1}' = x_m, \quad x_m' = x_1$$

with entry condition $P \equiv \bigwedge_i x_i = 0$, and candidates $c_i \equiv x_i \geq 0$. Each c_i is established. It is not preserved alone: a state can satisfy $x_i \geq 0$ while its predecessor under the cyclic assignment is negative. Nevertheless, the conjunction $\bigwedge_i x_i \geq 0$ is preserved because every successor coordinate is an old coordinate constrained by another conjunct. Thus all m candidates can fail standalone preservation while forming one valid, mutually supported invariant.

B.3 Greatest Jointly Inductive Candidate Subset

Let $C_0 = C$, and let $C_{j+1} \subseteq C_j$ be the set remaining after one ideal Houdini deletion round in Algorithm 1. Because C is finite, this descending sequence reaches the fixed point $\mathsf{H}_{\mathcal{L}}(C)$.

Proof of Theorem 3. At the fixed point, every $c \in \mathsf{H}_{\mathcal{L}}(C)$ satisfies establishment and preservation under the full conjunction $I_{\mathsf{H}_{\mathcal{L}}(C)}$. Applying Proposition 2 shows that this conjunction is valid.

It remains to prove greatestness. Let $C' \subseteq C$ be any jointly inductive subset. We show by induction over deletion rounds that $C' \subseteq C_j$. The base case $C' \subseteq C_0 = C$ is immediate. Assume $C' \subseteq C_j$, and take any $c \in C'$. Joint establishment of C' gives $P \Rightarrow c$, so c cannot be deleted for establishment. Joint preservation gives

$$\{I_{C'} \wedge B\}\, S\, \{c\}.$$

Since $C' \subseteq C_j$, the antecedent $I_{C_j} \wedge B$ implies $I_{C'} \wedge B$. Strengthening a valid Hoare precondition preserves validity, so

$$\{I_{C_j} \wedge B\}\, S\, \{c\}$$

also holds. Therefore no member of C' is deleted in round j, proving $C' \subseteq C_{j+1}$. At termination, $C' \subseteq \mathsf{H}_{\mathcal{L}}(C)$. Conjunction reverses set inclusion, hence $I_{\mathsf{H}_{\mathcal{L}}(C)} \preceq I_{C'}$. □

The theorem is candidate-relative: Houdini cannot synthesize a missing clause, so it need not recover the strongest invariant expressible in the full assertion language. It is also an ideal-oracle result: the inference-time Frama-C/WP backend may time out or fail to prove a valid obligation, in which case such clauses are conservatively removed. The final retained conjunction remains accepted by the verifier.

B.4 Closed Form of the Coverage-Game Shapley Value

For a fixed sampled full group $\mathcal{G}$, the allocation of Equation (1) is the Shapley value of $v_{\mathcal{G}}$. The group subscript is essential: the attributed sets $R_i(\mathcal{G})$ are formed only after the full union is filtered.

Proposition 4 (Closed form of the coverage game)**.** *For the fixed coverage game* $v_{\mathcal{G}}(\mathcal{S}) = |\bigcup_{i\in\mathcal{S}} R_i(\mathcal{G})|/|\mathcal{N}|$*, the Shapley value is* $\phi_i(\mathcal{G}) = \frac{1}{|\mathcal{N}|}\sum_{\nu\in R_i(\mathcal{G})} \frac{1}{f_{\mathcal{G}}(\nu)}$*, and* $\sum_{i=1}^{G} \phi_i(\mathcal{G}) = v_{\mathcal{G}}(\mathcal{G})$.

Proof. Fix a trace ν rejected by $f_{\mathcal{G}}(\nu)$ attributed rollout sets. In a uniformly random ordering of the group, ν contributes to the marginal gain of exactly one player—the first of those $f_{\mathcal{G}}(\nu)$ rollouts to appear—and by symmetry each is first with probability $1/f_{\mathcal{G}}(\nu)$. Player i therefore receives expected credit $1/f_{\mathcal{G}}(\nu)$ for each $\nu \in R_i(\mathcal{G})$ and zero otherwise. Summing over $\nu \in \mathcal{N}$ and normalizing by $|\mathcal{N}|$ gives the closed form; efficiency follows because each covered trace distributes exactly one unit of credit among its $f_{\mathcal{G}}(\nu)$ rejectors. □

C Implementation Details

Table 4 lists the implemented experimental settings. SFT synthesis, RL, and inference share one rollout–decompose–pool–filter–compose implementation. RL also filters the union once, then uses clause provenance to assign survivors back to their proposing rollouts before computing credit (Appendix L).

Programs for which the sampler retains no negative trace are unscorable and excluded from SFT. The exact near-input tier order is 0, 1, 2, -1, 3, 5, 8, -2, 10, 17, 4, -3, 25, 6, 40, 7, -5, 13, 50, 9.

Constant	Value	Role
runs requested	12	input executions; deterministic no-input programs collapse to one, and oracle-body sampling doubles the request
sampler seed	0	one canonical example set per reward request unless overridden
input tiers	20 fixed values	deterministic near-input schedule listed below; unconstrained tier candidates are clamped to $[-64, 64]$
far probes	3, plus 2 ordered multi-parameter probes	seed-hashed input-envelope coverage, magnitude ≤ 512
relation deltas	dense: $\pm\{1, 2\}$; terminal: $\pm\{1, 2, 3, 5, 8\}$	single-axis and pairwise perturbations; terminal bases require a witnessed final transition rather than a forward dense window
random deltas	$\pm[1, 34]$, then $\pm[35, 128]$ if needed	shared staged local support for the Random control and structured-budget fill
dense window	3	sampled successors required for a relation base
relation base cap	96	bases stratified across positives
post-exit steps	24	continuation length after a genuine exit (one negative trace per run)
negative-example budget	60 traces	up to 48 relational and 12 post-exit traces, then uniform random fill (sampler schema v9); the Random control uses 60 uniform traces
iteration cap	2×10^6	divergence safety net (capped runs flagged)
(w_c, w_g)	$(1.0, 0.3)$	negative rejection / Shapley group-credit weights
K	20 clauses	response cap applied independently to every generated response; later clauses are discarded
G_{SFT}	16 responses	group sampled per SFT strengthening round (§3.3)
T_{SFT}	5 rounds	maximum strengthening rounds per loop (at most 80 responses)
τ	0.5	SFT acceptance threshold on the composed set's trace-level empirical coverage
Houdini iterations	≤ 500	greatest-inductive-subset pruning
WP configuration	Alt-Ergo + Z3, 5 s, Typed model	per-obligation prover budget

Table 4: Reported settings for execution sampling (§3.2), SFT synthesis (§3.3), reward computation (§3.4), inference, and verification.

C.1 NEGATIVE-TRACE SAMPLING

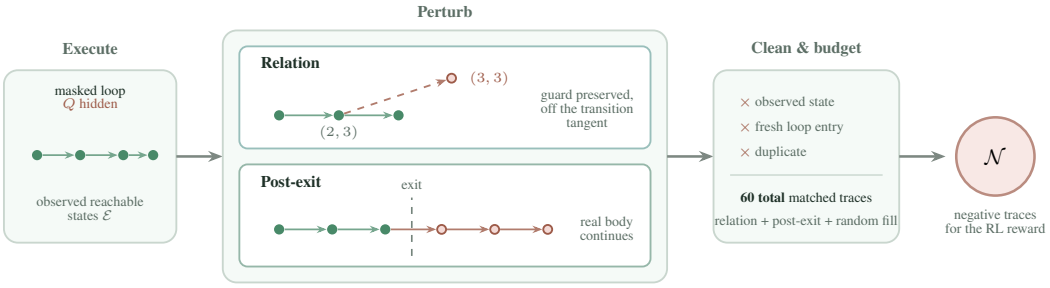

Figure 2: **Construction of target-hidden negative traces.** Concrete executions provide observed reachable states. Two transformations probe relational structure and behavior past a genuine exit. Conservative cleaning selects these traces first, then random perturbations fill the common 60-trace budget. The resulting $\mathcal{N}$ is used for SFT acceptance and RL reward computation and never exposes Q.

Algorithm 2 summarizes the sampler used for RL. Trace records real loop-head states and, after each genuine exit, continues the real loop body to obtain one post-exit trace. $\mathrm{Perturb}_\Theta$ applies the configured single-variable and pairwise changes; the exact perturbations and budgets are those in Table 4.

The two structured families probe complementary properties. A relation perturbation is admitted only when it stays inside the sampled envelope of its context, preserves the guard truth value, and does not lie on the locally witnessed transition tangent, so only clauses that constrain relations among variables can reject it. A post-exit continuation executes the real body past a witnessed exit into states that no perturbation of an observed state can produce, so only clauses that pin down the exit boundary reject it. After cleaning and structural selection, uniform local perturbations fill the unused portion of the 60-trace budget. The Random control draws all 60 traces from that same generator, with the same seed, reachability checks, fresh-entry filter, and deduplication rule.

Algorithm 2 Target-hidden negative-trace sampling

Input: masked loop $\mathcal{L} = (P, B, S)$, input schedule U, configuration Θ
Output: reachable states $\mathcal{E}$, negative traces $\mathcal{N}$

$\mathcal{T}_{\mathrm{exec}}, \mathcal{T}_{\mathrm{post}}, \textit{capped} \leftarrow \mathrm{Trace}(\mathcal{L}, U, \Theta)$ ▷ *capped: some run was capped*
$\mathcal{E} \leftarrow \mathrm{UniqueHeads}(\mathcal{T}_{\mathrm{exec}})$; $M \leftarrow \mathrm{SafeModifiedVars}(S)$
$\mathcal{E}_{\mathrm{sample}} \leftarrow \mathrm{StratifiedSample}(\mathcal{E})$
$a \leftarrow \mathrm{RelationSafe}(S)$; $Z_{\mathrm{rel}} \leftarrow \varnothing$; **if** a **then** $Z_{\mathrm{rel}} \leftarrow \mathrm{Perturb}_\Theta(\mathrm{Dense}(\mathcal{E}_{\mathrm{sample}}), M)$
if a and $\neg$*capped* **then** $Z_{\mathrm{rel}} \leftarrow Z_{\mathrm{rel}} \cup \mathrm{Perturb}_\Theta(\mathrm{Terminals}(\mathcal{T}_{\mathrm{exec}}), M)$
$Z_{\mathrm{rel}} \leftarrow \mathrm{TangentAndEnvelopeFilter}(Z_{\mathrm{rel}}, \mathcal{T}_{\mathrm{exec}})$
$(Z_{\mathrm{rel}}, \mathcal{T}_{\mathrm{post}}) \leftarrow \mathrm{Clean}_{\mathcal{E}}(Z_{\mathrm{rel}}, \mathcal{T}_{\mathrm{post}})$
$\widehat{Z}_{\mathrm{rel}}, \mathcal{N}_{\mathrm{post}} \leftarrow \mathrm{BudgetAndRoundRobin}(Z_{\mathrm{rel}}, \mathcal{T}_{\mathrm{post}}, \Theta)$
$\mathcal{N}_{\mathrm{core}} \leftarrow \{\langle s \rangle : s \in \widehat{Z}_{\mathrm{rel}}\} \cup \mathcal{N}_{\mathrm{post}}$; $b \leftarrow 60 - |\mathcal{N}_{\mathrm{core}}|$
$Z_{\mathrm{rand}} \leftarrow \mathrm{UniformLocalPerturb}(\mathcal{E}_{\mathrm{sample}}, M, b)$
$\widehat{Z}_{\mathrm{rand}} \leftarrow \mathrm{CleanAndBudget}_{\mathcal{E}}(Z_{\mathrm{rand}}, b; \mathcal{N}_{\mathrm{core}})$
$\mathcal{N} \leftarrow \mathcal{N}_{\mathrm{core}} \cup \{\langle s \rangle : s \in \widehat{Z}_{\mathrm{rand}}\}$
return $(\mathcal{E}, \mathcal{N})$

$\mathrm{Clean}_{\mathcal{E}}$ removes observed reachable states, possible fresh loop entries, duplicates, and unsupported candidates; a cleaned post-exit continuation remains one trace, and relation candidates become singleton traces. Within each structural bucket the round-robin prefers the smallest counterfactual change, so surviving witnesses sit close to the sampled manifold. Random fill begins only after the structured families are selected and shares their cross-family duplicate set. Budget matching counts traces: a multi-state post-exit continuation is one trace, just as it is one rejection event in the reward.

C.2 Group-Relative Optimization Details

For a sampled response group $Y_{1:G} \sim \pi_{\theta_{\text{old}}}(\cdot \mid \mathcal{L})$, we normalize Equation (2) within its group,

$$\widehat{\text{Adv}}_i = \frac{r_i(\mathcal{G}) - \text{mean}_{j \in \mathcal{G}}\, r_j(\mathcal{G})}{\text{std}_{j \in \mathcal{G}}\, r_j(\mathcal{G}) + 10^{-6}},$$

and use the clipped GRPO objective (Shao et al., 2024)

$$\mathcal{J}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{T_i}\sum_{u=1}^{T_i}\left\{\min\Big(\rho_{i,u}\widehat{\text{Adv}}_i, \text{clip}(\rho_{i,u}, 1-\varepsilon, 1+\varepsilon)\widehat{\text{Adv}}_i\Big) - \beta\widehat{d}_{i,u}\right\}\right]. \quad (3)$$

Here T_i is the token length of response i, and the token-level importance ratio is

$$\rho_{i,u} = \frac{\pi_\theta(y_{i,u} \mid y_{i,<u}, \mathcal{L})}{\pi_{\theta_{\text{old}}}(y_{i,u} \mid y_{i,<u}, \mathcal{L})}.$$

Writing $h_{i,u} = (\mathcal{L}, y_{i,<u})$, the implementation uses the per-token sampled estimator of the forward conditional KL,

$$\delta_{i,u} = \log \pi_{\text{ref}}(y_{i,u} \mid h_{i,u}) - \log \pi_\theta(y_{i,u} \mid h_{i,u}), \qquad \widehat{d}_{i,u} = \exp(\delta_{i,u}) - \delta_{i,u} - 1.$$

The KL penalty is therefore evaluated at each sampled token and its explicit prefix context, then averaged with the token objective; it is neither an unnormalized sequence KL nor an unspecified sequence-level penalty. The clipping radius is $\varepsilon > 0$, the KL coefficient is $\beta = 0.001$, and π_{ref} is frozen.

The canonical normalization key used for de-duplication applies only these finite rules: strip the annotation wrapper and redundant whitespace; parse supported scalar-integer expressions so parentheses disappear; orient $>$ and $\geq$ as $<$ and $\leq$; sort the two sides of $=$ and $\neq$; eliminate unary plus and double negation; rewrite negated comparisons to their complementary comparison; sort the immediate operands of addition and multiplication; remove $+0$, -0, and $\times 1$; flatten association and remove exact duplicates within $\wedge$ and $\vee$; and expand chained comparisons into conjunctions. It does not reorder Boolean operands, normalize general arithmetic, cancel terms, invoke SMT, or otherwise decide logical equivalence. Unsupported expressions use their whitespace-normalized text as the key. Pooled survivors are assigned back by this same canonical normalization key, so only spellings identified by these rules share a pooled entry and credit.

A zero-coverage support clause receives zero direct reward even if it enables another clause to survive; the objective has no support-aware term. Both coverage and Shapley credit lie in $[0, 1]$, so the full reward lies in $[0, 1.3]$. When the sampler retains no negatives, scoring falls back to binary inductiveness for a nonempty response: it receives one iff every admitted normalized clause is assigned back from the pooled survivor set, and zero otherwise.

D Generation Prompt

The canonical evaluation template appears below; every target and non-contract comment is stripped before insertion, and general annotation rules are supplied by the accompanying system prompt. Both training sets are rebuilt with this single prompt pair: archival records are sanitized to the scalar-integer single-loop interface (37,481 RL and 3,200 SFT records retained), and no model-visible prompt contains a target clause or non-contract comment. The released training sets are further curated from this sanitized pool (Appendix G).

Generation prompt

```
You are given a C function. Produce the strongest inductive ACSL loop
invariants that capture the loop's behavior (conservation/relational laws
+ tight progress bounds). Output ONLY invariant lines, each formatted
exactly as:
loop invariant <expr>;
Output at most 20 invariant lines.

Program:
{program}
```

The accompanying system prompt appears verbatim below.

System prompt

```
You are a formal verification expert specializing in ACSL loop invariant
synthesis for Frama-C/WP.

## LOOP INVARIANT DEFINITION

A loop invariant is a property that:
1. Holds before the first loop iteration (initialization).
2. Is preserved by every loop iteration when the loop guard is true
   (inductiveness).
3. Combined with loop exit (`!guard`), helps establish required
   loop-exit facts.

## RULES

- Add one core conservation/relational equality that explains loop
  updates (state transition law).
- Add minimal progress bounds for loop counters and key arithmetic
  variables.
- Prefer strong invariants; avoid weak/tautological ones.
- Do not add unrelated invariants just to increase count.
- MUST NOT modify or rewrite any `unknown()`/`unknownN()` call.
- `\at(v, Pre)` can ONLY be used with function parameters, never with
  local variables initialized inside the function.
- `\at(v, LoopEntry)` denotes the value of v at the start of the loop
  and can ONLY be used with local variables initialized before the
  loop (using v = unknown();).
- Loop guard is NOT an invariant; weaken strict guards to non-strict
  bounds at exit.
- Parenthesize mixed equality and relational expressions so their
  grouping is explicit.
- Use `loop invariant` only as a line prefix, never inside expressions.
- Emit scalar invariant expressions only; do not declare predicates,
  logic functions, axioms, or lemmas.
- Do NOT use the ternary `? :` operator; split cases with `==>` instead.
- `^` is bitwise XOR in C/ACSL, not exponentiation --- write `x*x*x`
  for x cubed.
- Do NOT place a C boolean directly in arithmetic (`x + (a > b)` is
  invalid); use `==>` to split cases.
- Only use variables declared in the given function.
- Operators: comparison `==` `!=` `<` `<=` `>` `>=`; logical `&&` `||`
  `!` `==>` `<==>`; arithmetic `+` `-` `*` `/` `%`.

## ACSL INVARIANT SYNTAX

Allowed expression forms (all verified by Frama-C/WP):

1. **Bounds**: `0 <= i`, `i <= n`, `x >= 1`
2. **Linear equality**: `i + c == n`, `2 * s == i * (i - 1)`,
   `x == q * y + r`
3. **Polynomial equality**: `x == n * n * n`,
   `6 * s == i * (i + 1) * (2 * i + 1)`
4. **Relational identity** (multi-var algebraic law):
   `1 + x * z - x - z * y == 0`, `p * s - r * q == 1`
5. **Implication**: `i > 0 ==> s >= i`, `a != 0 ==> q >= 1`
6. **Modular**: `i % 5 == 0`, `r >= 0 && r < y`
7. **Bitshift** (power-of-2 only): `p == 1 << i`, `s == (1 << i) - 1`
8. **Conjunction**: `c >= 1 && c <= k`
9. **\at for params**: `n == \at(n, Pre)` (function parameters only)

Forbidden:
- `^` for exponentiation (it is XOR) --- use repeated `*` or `<<` for
  powers of 2.
- `\product`, `\sum`, `\factorial` --- not ACSL scalar operators.
- `? :` ternary --- rewrite with `==>`.
- `\old(x)` --- use `\at(x, Pre)`.
- Type casts `(int)`, `(long)` --- unnecessary in ACSL logic.
- Function calls, predicates, lemmas, axioms --- emit only scalar
  expressions.

Operators (complete list):
  comparison: `==` `!=` `<` `<=` `>` `>=`
  logical:    `&&` `||` `!` `==>` `<==>`
  arithmetic: `+` `-` `*` `/` `%` `<<` `>>`

## OUTPUT

Return ONLY invariant lines, one per line, formatted exactly as:
`loop invariant <property>;`
Do not return code fences, `loop assigns`, the surrounding C function,
or explanations.
```

E TRAINING INTERFACE

The training service is packaged as a self-contained, CPU-only container with gcc, the verifier, Why3, and its prover backend. It exposes POST /reward, accepting program source, an array of model responses, reward_variant in {binary, whole_coverage, base, full}, credit_filter_order in {pooled, independent}, and sampler.negative_sampler in {random, structured}. These switches default to full, pooled, and structured, respectively; independent is retained only for historical comparison. The response reports each rollout's reward together with its coverage and Shapley-credit components. Responses may be raw LLM text, invariant lists, or annotated code; an offline scorer provides the same generation reward over JSONL/Parquet rollout dumps.

F Training–Evaluation Overlap Audit

We compare the final RL (4,992 records and 4,992 distinct program sources) and SFT (33,346 records and 31,865 distinct program sources) training sets against all 832 evaluation programs after target removal. Under three increasingly aggressive normalizations—exact text, alpha-renamed identifiers, and alpha-renaming with integer constants abstracted—no training program matches an evaluation program (Table 5).

We separately measure coarse structural overlap. A structural cell combines the control-flow profile, guard kind, nonlinearity, nondeterminism, and a variable-count band. The evaluation set occupies 122 such cells. RL and SFT share 88 and 41 of them, respectively; their union shares 92 cells. Equivalently, 702 (84.4%), 529 (63.6%), and 754 (90.6%) of the 832 evaluation programs fall in a cell represented by RL, SFT, and their union. These are support statistics, not instance matches. The record-weighted total-variation distances over cells are 0.422 for RL, 0.651 for SFT, and 0.596 for their union. Table 6 gives interpretable feature marginals behind this joint-cell comparison.

	RL	SFT	RL ∪ SFT
Records	4,992	33,346	38,338
Distinct program sources	4,992	31,865	35,170
Exact-text matches	0	0	0
Alpha-renamed matches	0	0	0
Alpha-renamed, constant-abstracted matches	0	0	0
Shared evaluation cells	88 / 122	41 / 122	92 / 122
Evaluation programs in shared cells	702 (84.4%)	529 (63.6%)	754 (90.6%)
Training records in evaluation-supported cells	4,992 (100%)	27,275 (81.8%)	32,267 (84.2%)
TV distance over structural cells	0.422	0.651	0.596

Table 5: Instance and structural-distribution audit of the final training sets against the 832 target-removed evaluation programs. Distribution statistics count training records, matching their sampling frequency.

Feature (% of records/programs)	Evaluation	RL	SFT
Constant guard	24.6	7.5	4.6
Single-variable guard	31.4	31.6	44.0
Variable-vs.-variable guard	37.3	57.3	50.6
Compound guard	6.7	3.7	0.8
Nonlinear	2.5	2.5	10.8
Nondeterministic	28.6	30.3	14.9
≤ 2 variables	35.1	21.2	46.2
3–4 variables	33.9	31.7	12.0
≥ 5 variables	31.0	47.1	41.8

Table 6: Marginal structural distributions of evaluation programs and final training records. The structural-cell TV distances in Table 5 use the joint features rather than these marginals independently.

Scope of the audit. The audit detects direct source reuse under the three reported normalizations; it does not establish semantic inequivalence between arbitrarily rewritten programs or address possible benchmark exposure during foundation-model pretraining. Structural-cell overlap and total-variation distance characterize domain similarity.

G Training-Pool Curation

The released training sets are filtered and synthesized from the 37,481-record sanitized pool of Appendix D: programs with no training signal and too-easy or too-hard programs are removed, the selection is aligned with the evaluation distribution over structural cells (control-flow profile, guard

kind, nonlinearity, nondeterminism, variable band), and SFT targets are minimal inductive invariant sets synthesized with GPT-5.6 Luna and verified by Frama-C/WP. Per-stage counts are emitted as machine-readable reports shipped with the code. Table 7 summarizes the released sets.

RL set	4,992 records over 1,131 distinct loop shapes
SFT set	33,346 records over 31,865 distinct program sources
RL–SFT exact-source overlap	1,687 distinct programs

Table 7: Released training sets.

The reward discriminates on the released data. On the 125 released RL programs with a reference answer, the production reward gives the reference a median of 0.845 and its bounds-only projection a median of 0.0; the tautology 1 == 1, the loop guard copied as an invariant, and an unsound clause score exactly zero on every program. A simulated eight-answer group has a median within-group reward standard deviation of 0.21, so group-relative updates retain a usable advantage signal.

H Experimental Protocol Details

The evaluation workload composition is given in Section 4.1; the 466 Loopy programs are the integer subset of its original 469-program corpus (three floating-point programs excluded), and unsupported inputs remain failures in the common denominator.

Before model invocation, we remove assert and ensures predicates, executable assertion calls, conventional error-label names, and non-contract comments, including source-provenance paths; preconditions and executable semantics remain visible. CRAFT evaluation runs use the canonical generation template, extractor, semantic deduplication, and Houdini implementation; external tools retain their own orchestration. Each model retains its serving interface and chat template. In CRAFT, API calls set reasoning_effort=none; all locally served checkpoints disable thinking. We set temperature 1.0 and top-p 1.0, without an additional top-k, repetition penalty, re-roll, or target feedback. The cap is 8,192 completion tokens for API models, including any provider-internal reasoning tokens, and 2,048 emitted tokens for local vLLM models. Each RQ1 curve reuses the same saved response pool per task.

Training configuration. Tables 8 and 9 record the supplied SFT and RL run configurations. Reward and sampler constants are reported separately in Table 4.

Field	Value
Learning rate	1×10^{-5}
Epochs	3
Batch (per device $\times$ accumulation $\times$ GPUs)	global batch 32; $2 \times 2 \times 8$ or $1 \times 4 \times 8$
Sequence cutoff	2,048
DeepSpeed	ZeRO-3
Scheduler	cosine

Table 8: SFT training configurations.

Field	Value
Batch size	64
Learning rate	1×10^{-6}
KL penalty	low_var_kl, coefficient 0.001
Rollouts per prompt	8
Tensor parallelism	1–2
GPU memory utilization	0.3–0.5
Epochs	5
Total steps	200
Checkpoint interval	100

Table 9: RL training configurations.

I Complete Experimental Data

This section collects the per-model and per-stratum measurements behind the main aggregates.

I.1 Probe Curves

Table 10: Complete base-checkpoint probe results by benchmark stratum and sampling budget. Each cell reports a percentage; All / 832 is the whole-workload result. k_{95} is the smallest measured budget at which compose@k reaches 95% of compose@32. Bold marks the best value in each column at a fixed budget.

		Linear / 316		NLA / 50		Loopy / 466		All / 832		
Model	k	pass	comp	pass	comp	pass	comp	pass	comp	k_{95}
Qwen3-1.7B	1	7.6	22.2	2.4	2.0	8.4	25.3	7.8	22.7	32
	4	15.3	33.9	5.0	8.0	15.9	35.2	15.0	33.1	
	8	19.4	37.7	**6.1**	8.0	19.7	39.3	18.8	36.8	
	16	23.2	39.2	**7.3**	10.0	23.5	42.5	22.4	39.3	
	32	26.6	41.5	**8.0**	10.0	27.5	44.6	26.0	41.4	
Qwen3-4B	1	6.2	38.6	0.5	**8.0**	7.2	41.4	6.4	38.3	16
	4	11.7	45.9	1.7	8.0	12.7	50.6	11.6	46.3	
	8	14.1	51.9	3.0	8.0	15.2	55.4	14.1	51.2	
	16	16.6	55.1	4.5	10.0	17.7	59.9	16.5	55.0	
	32	19.3	57.0	6.0	**14.0**	20.4	61.6	19.1	57.0	
Qwen3-8B	1	8.6	43.7	3.2	6.0	8.6	47.9	8.3	43.8	16
	4	16.8	56.6	5.3	6.0	17.4	59.4	16.5	55.2	
	8	**20.7**	60.4	5.9	12.0	22.0	63.1	20.5	59.0	
	16	**24.5**	61.7	6.0	12.0	26.2	66.1	24.4	61.2	
	32	27.8	63.6	6.0	12.0	30.7	67.2	28.1	62.5	
Qwen3-14B	1	8.9	**54.1**	1.1	**8.0**	**11.8**	**56.2**	**10.1**	**52.5**	8
	4	15.7	**64.6**	3.4	**10.0**	**20.2**	**66.1**	**17.5**	**62.1**	
	8	19.3	**68.0**	4.9	12.0	**24.3**	**68.7**	**21.3**	**65.0**	
	16	23.6	**69.3**	5.9	12.0	**28.3**	**70.0**	**25.2**	**66.2**	
	32	**28.2**	**71.5**	6.0	12.0	**32.2**	**71.2**	**29.1**	**67.8**	
Qwen3-30B-A3B	1	**10.6**	48.1	**3.6**	**8.0**	9.8	52.4	9.7	48.1	8
	4	**17.0**	57.6	**5.7**	8.0	17.9	65.0	16.9	58.8	
	8	19.7	61.1	6.0	8.0	21.9	67.2	20.1	61.3	
	16	22.5	62.0	6.0	10.0	26.2	68.2	23.6	62.4	
	32	25.6	62.3	6.0	10.0	30.5	68.5	27.2	62.6	
Llama 3.1 8B	1	0.8	29.4	0.0	2.0	0.9	32.0	0.8	29.2	8
	4	3.0	49.7	0.0	8.0	3.3	51.7	3.0	48.3	
	8	5.5	55.4	0.0	**14.0**	5.9	58.4	5.4	54.6	
	16	9.3	57.0	0.0	**14.0**	9.6	59.9	8.9	56.0	
	32	14.6	57.0	0.0	**14.0**	14.8	60.1	13.8	56.1	

Table 11: Per-GPU decoding throughput of the four backbones used in training experiments, measured with vLLM on A800-80G GPUs (n=32 responses per task, temperature 1.0, max_tokens=2048).

Model	tok/s/GPU
Qwen3-4B	7,445
Qwen3-8B	5,833
Qwen3-14B	3,168
Llama 3.1 8B	5,135

I.2 SFT Probe Curves

The SFT checkpoints (three epochs on the final SFT corpus) are evaluated under the same 832-task protocol as the base probe: 32 saved target-hidden responses per task at temperature 1.0, 5-second per-obligation prover budget, and the current prompt.

Table 13: Complete SFT probe results on All / 832. k_{95} is the smallest measured $k \in \{1, 4, 8, 16, 32\}$ at which compose@k reaches 95% of its value at $k = 32$. Bold marks the best value in each column.

Whole-response pass rates

Model	pass@1	pass@4	pass@8	pass@16	pass@32
Qwen3-4B + SFT	40.43	55.23	59.99	64.09	67.74
Qwen3-8B + SFT	41.93	56.82	61.11	64.68	67.98
Qwen3-14B + SFT	**45.66**	**59.14**	**63.50**	**67.34**	**70.57**
Llama 3.1 8B + SFT†	36.20	51.00	55.80	60.00	63.20

Clause-composition rates

Model	compose@1	compose@4	compose@8	compose@16	compose@32	k_{95}
Qwen3-4B + SFT	62.62	73.44	76.20	77.28	77.88	8
Qwen3-8B + SFT	65.50	75.12	77.52	78.61	78.97	4
Qwen3-14B + SFT	**66.35**	**76.68**	**78.37**	**79.21**	**79.33**	4
Llama 3.1 8B + SFT†	56.80	69.00	72.50	73.40	74.00	8

Here and in the main experimental table, † marks provisional mock values used for presentation layout; these entries must be replaced by completed measurements before submission.

Table 14: Per-GPU decoding throughput of the SFT checkpoints, measured with vLLM on A800-80G GPUs (n=100 responses per task, temperature 1.0, max_tokens=2048).

Model	tok/s/GPU
Qwen3-4B + SFT	7,445
Qwen3-8B + SFT	5,833
Qwen3-14B + SFT	3,168
Llama 3.1 8B + SFT	5,135

	Linear / 316		NLA / 50		Loopy / 466		All / 832	
Stage	k=1	k=8	k=1	k=8	k=1	k=8	k=1	k=8
Qwen3-4B								
Zero, no pipeline (pass)	6.2	14.1	0.5	3.0	7.2	15.2	6.4	14.1
Zero + pipeline (compose)	38.6	51.9	8.0	8.0	41.4	55.4	38.3	51.2
+SFT	65.8	81.0	8.0	12.0	66.3	79.8	62.6	76.2
+SFT+RL †	70.6	83.5	10.0	14.0	71.2	84.7	67.3	80.0
SFT, no pipeline (pass)	43.7	63.4	2.1	6.2	42.3	63.5	40.4	60.0
SFT+RL, no pipeline (pass) †	42.8	61.8	2.0	6.0	40.2	59.6	38.9	57.2
Qwen3-8B								
Zero, no pipeline (pass)	8.6	20.7	3.2	5.9	8.6	22.0	8.3	20.5
Zero + pipeline (compose)	43.7	60.4	6.0	12.0	47.9	63.1	43.8	59.0
+SFT †	69.3	82.6	10.0	16.0	68.9	80.7	65.5	77.5
+SFT+RL †	76.5	86.0	12.0	18.0	75.7	85.4	72.2	81.6
SFT, no pipeline (pass) †	46.0	65.0	3.0	8.0	43.4	64.2	41.9	61.1
SFT+RL, no pipeline (pass) †	44.8	64.5	3.0	8.0	42.1	63.1	40.8	60.3
Qwen3-14B								
Zero, no pipeline (pass)	8.9	19.3	1.1	4.9	11.8	24.3	10.1	21.3
Zero + pipeline (compose)	54.1	68.0	8.0	12.0	56.2	68.7	52.5	65.0
+SFT	70.3	82.3	12.0	16.0	69.5	82.4	66.4	78.4
+SFT+RL †	74.5	86.0	14.0	20.0	74.4	85.1	70.8	81.5
SFT, no pipeline (pass)	50.2	68.0	3.5	9.8	47.1	66.2	45.7	63.5
SFT+RL, no pipeline (pass) †	49.0	67.0	3.5	9.0	46.4	65.9	44.8	62.9
Llama 3.1 8B								
Zero + pipeline (compose)	29.4	55.4	2.0	14.0	32.0	58.4	29.2	54.6
+SFT †	60.0	78.0	8.0	16.0	59.9	74.8	56.8	72.5
SFT, no pipeline (pass) †	40.0	60.0	2.0	6.0	37.3	58.3	36.2	55.8

Table 12: Cumulative contribution of each pipeline component, by benchmark stratum (% verified, reasoning_effort=none/non-thinking). Framework-free rows report pass@k; pipeline rows report compose@k on the same saved responses, so adjacent rows isolate one component. RL is trained on the three Qwen3 models; Llama 3.1 8B reports SFT rows only. Complete probe curves and per-stratum results for all six models appear in Table 10. API models under the same protocol appear in Table 19 (Appendix I.6). † denotes provisional mock values, not completed measurements.

I.3 Reinforcement-Learning Results

Table 15: Complete comparison before and after RL. The Zero and SFT initializations are evaluated under the same decoding and verification configuration.

Checkpoint	Stage	pass@1	compose@1	compose@8	compose@16	compose@32	k_{95}
Zero	before RL	8.3%	43.8%	59.0%	61.2%	62.5%	16
RL-Zero	after RL	6.8%	57.9%	71.3%	73.4%	74.3%	8
SFT	before RL	41.9%	65.5%	77.5%	78.6%	79.0%	4
SFT+RL†	after RL	40.8%	72.2%	81.6%	82.4%	82.7%	4

The † row is a provisional mock curve, not an empirical result.

Table 16: Provisional seed-level SFT+RL results on Qwen3-8B (All / 832, %). All entries are mock values and therefore red. The mean is the curve used in Table 15 and Figure 3.

Run	pass@1	compose@1	compose@4	compose@8	compose@16	compose@32
Seed 1	41.3	71.8	78.9	81.0	81.9	82.1
Seed 2	40.2	72.6	79.8	82.0	82.8	83.1
Seed 3	40.9	72.2	79.5	81.8	82.5	82.9
Mean	40.8	72.2	79.4	81.6	82.4	82.7
Std. dev.	0.6	0.4	0.5	0.5	0.5	0.5

I.4 Reward and Negative-Sampler Ablations

The reward variants correspond directly to the two levels of Equation (2). Define $\mathrm{NKeys}(C) = \{\mathrm{nkey}(c) : c \in C\}$ and

$$\chi_i(\mathcal{G}) = \mathbf{1}\big[C_i^{\mathrm{raw}} \neq \varnothing \ \wedge \ \mathrm{NKeys}(C_i^{\mathrm{attr}}(\mathcal{G})) = \mathrm{NKeys}(C_i^{\mathrm{raw}})\big] .$$

Binary Inductiveness uses $r_i^{\mathrm{bin}}(\mathcal{G}) = \chi_i(\mathcal{G})$: it assigns one iff the capped, normalized response is nonempty and every admitted normalization key is assigned back from the pooled survivor set. *Whole-Rollout Strength* pays $\mathrm{cov}_i(\mathcal{G})$ only under that same condition,

$$r_i^{\mathrm{whole}}(\mathcal{G}) = \chi_i(\mathcal{G}) \frac{|R_i(\mathcal{G})|}{|\mathcal{N}|}.$$

Clause-Decomposed Strength removes this all-or-nothing indicator and scores $C_i^{\mathrm{attr}}(\mathcal{G})$; the *Full Compositional Credit* additionally adds $w_g \phi_i(\mathcal{G})$. These are binary, whole_coverage, base, and full, respectively, abbreviated Binary, Whole-rollout, Clause-decomposed, and Full in figures and prose. All variants train from the Zero initialization on the curated pool with identical optimization and inference settings. The Full Compositional Credit run is the RL-Zero checkpoint of Section 4.5.

Table 17: Reward-function ablation on Qwen3-8B, trained from the Zero initialization (%); complete pass@k and compose@k curves. Best result per column among trained variants in bold.

	pass@k					compose@k				
Variant	1	4	8	16	32	1	4	8	16	32
Zero (untrained reference)	8.3	16.5	20.5	24.4	28.1	43.8	55.2	59.0	61.2	62.5
Binary Inductiveness	9.88	13.33	15.11	17.08	19.30	8.77	15.62	20.19	24.04	27.88
Whole-Rollout Strength	**25.35**	**28.10**	**29.07**	**30.10**	**31.32**	27.76	29.21	29.93	31.13	33.17
Clause-Decomposed Strength	4.60	10.16	13.31	16.27	19.03	**58.29**	65.99	67.79	69.35	70.19
Full Compositional Credit (ours)	6.80	13.42	16.80	20.17	23.44	57.93	**67.19**	**71.27**	**73.44**	**74.28**

The negative-sampler ablation uses matched Qwen3-8B runs from the Zero initialization under the full compositional reward that differ only in the sampler; the Structured row is the Full Compositional Credit run of Table 17. Structured is the sampler of Section 3.2: it selects relation and post-exit traces first, then fills unused slots with uniform local perturbations. Random draws all 60 traces from the same uniform generator.

Table 18: Negative-sampler ablation on Qwen3-8B, trained from the Zero initialization under the full reward (%). Bold marks the better sampler per column.

	pass@k					compose@k				
Negative sampler	1	4	8	16	32	1	4	8	16	32
Random†	**8.63**	**14.40**	**17.00**	19.43	21.82	52.04	60.46	64.54	67.31	68.75
Structured (ours)	6.80	13.42	16.80	**20.17**	**23.44**	**57.93**	**67.19**	**71.27**	**73.44**	**74.28**

The † control is a mock row pending the matched reward configuration. It yields a compose@k difference of 5.5–6.7 points at every budget, while pass@k stays within -1.8 to $+1.6$ points of Random.

I.5 Does Structured Negative Coverage Predict Target Success?

This audit evaluates whether the *negative sampler's* target-independent coverage score ranks candidate sets by their eventual hidden-target utility. We collect 6,190 saved candidate invariant sets produced by eight target-hidden generation pipelines on the 832-program test set. For each candidate, the prediction score is its structured negative coverage, including random budget fill and computed without Q; the binary label is the final Frama-C/WP verdict after the untouched target is restored. No target label is used to fit a predictor or choose a threshold.

The continuous analysis contains 6,182 candidates from 831 programs. The remaining eight candidates come from one program whose loop variable is uninitialized and whose guard is controlled by an unknown value; every local perturbation remains a possible fresh entry, so neither sampler mode admits a negative trace. Because multiple candidate generators are evaluated on the same program, confidence intervals use 5,000 stratified program-cluster bootstrap replicates: all candidates for a resampled program move together. A second analysis computes AUROC separately within each of the 593 programs having both successful and unsuccessful candidates, then macro-averages over programs. This removes between-program difficulty as an explanation for the ranking.

Population	Candidates	Positive rate	AUROC	AUPRC
All	6,182	0.525	0.793	0.768
Linear	2,520	0.544	0.821	0.826
NLA	400	0.090	0.554	0.132
Loopy	3,262	0.564	0.798	0.796

The pooled AUROC has a program-clustered 95% interval of $[0.775, 0.811]$, and the AUPRC interval is $[0.742, 0.794]$, compared with the pooled positive-rate baseline of 0.525. Within programs, macro AUROC is 0.802, with a clustered interval of $[0.783, 0.820]$; randomly permuting target verdicts among candidates for the same program gives a one-sided $p < 2 \times 10^{-4}$ over 5,000 permutations. Method-stratified AUROCs range from 0.574 to 0.820. Candidates with coverage at least 0.90 verify 80.1% of the time, compared with 22.4% below 0.50.

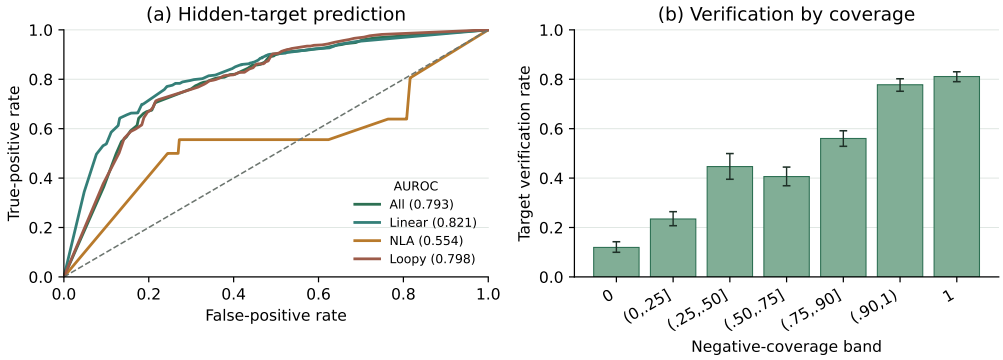

Figure 4: Predictive validity of target-independent negative coverage on saved test-set candidates. (a) ROC curves for restored-target verification; numbers in parentheses are AUROCs. (b) Target verification rate by coverage band, with Wilson 95% intervals. NLA remains close to chance, showing that the current structured negatives discriminate hidden-target utility much better for Linear and Loopy programs than for nonlinear arithmetic.

Coverage provides a useful ranking signal for candidate strength. The NLA stratum identifies where richer nonlinear perturbations are needed.

I.6 Cross-Model Main Results

Table 19: Complete reasoning-off cross-model results by benchmark stratum and sampling budget. Each cell reports a percentage; All / 832 is the whole-workload result. Rows for each model come from one archived rollout pool of ten responses per task. compose@k uses prefix composition and pass@k uses the unbiased estimator. Bold marks the best value in each column at a fixed budget. GPT-5-mini is excluded because it has no reasoning-off setting.

		Linear / 316		NLA / 50		Loopy / 466		All / 832	
Model	k	pass	comp	pass	comp	pass	comp	pass	comp
GPT-5-nano	1	3.77	34.81	0.40	22.00	3.80	34.55	3.58	33.89
	4	10.79	48.10	1.60	30.00	11.48	51.72	10.62	49.04
	8	15.84	55.38	3.20	44.00	17.58	56.87	16.05	55.53
GPT-5	1	**35.06**	52.22	5.80	36.00	33.78	52.36	32.58	51.32
	4	**47.73**	64.87	8.74	52.00	**50.67**	67.38	**47.03**	65.50
	8	**51.93**	70.25	**9.60**	66.00	**56.13**	73.82	**51.74**	72.00
Claude Sonnet-4.6	1	34.49	**55.38**	**7.80**	**44.00**	**40.67**	**59.44**	**36.35**	**56.97**
	4	40.32	62.97	**8.80**	52.00	47.27	66.31	42.32	64.18
	8	42.87	66.46	**9.60**	64.00	50.12	67.60	44.93	66.95
DeepSeek V4-Flash	1	28.67	52.22	5.00	**44.00**	30.15	51.72	28.08	51.44
	4	42.35	**66.46**	7.52	**60.00**	47.73	**69.96**	43.27	**68.03**
	8	46.64	**71.52**	8.00	**72.00**	53.59	**76.61**	48.21	**74.40**

Each pass@k/compose@k pair at a given k is certified from the same saved k responses.

I.7 Comparison with Specialized Tools

Table 20: Archived target-hidden tool comparison: the reasoning_effort=none rows from the main text alongside the archived reasoning_effort=medium rerun; Daikon makes no model call and is listed once. Each cell reports the verification rate for Linear, NLA, Loopy, or all 832 programs; the Loopy column is the benchmark stratum, whereas the Loopy row is the tool of Kamath et al. (2023). The compose@1 rows are the dedicated single-call runs paired with pass@1. Medium-effort results are available only for these single-response measurements; the compose@4 and compose@8 rows use reasoning_effort=none and are prefix compositions of the archived ten-rollout pool, as in Table 2.

Method	Reason.	Linear	NLA	Loopy	All	Tokens/task	Time/task
AutoSpec	none	47.78%	6.00%	42.27%	42.19%	2,181.72	27.34 s
AutoSpec	medium	65.82%	8.00%	64.81%	61.78%	25,154.54	54.77 s
SESpec	none	28.80%	4.00%	33.05%	29.69%	22,081.61	68.50 s
SESpec	medium	56.96%	6.00%	49.57%	49.76%	44,807.05	117.31 s
Clause2Inv	none	20.89%	0.00%	0.00%	7.93%	1,056.25	8.41 s
Clause2Inv	medium	19.94%	2.00%	0.00%	7.69%	385.80	29.04 s
Daikon	—	23.10%	0.00%	21.67%	20.91%	0	4.89 s
Loopy	none	20.25%	0.00%	19.74%	18.75%	14,513.37	147.16 s
Loopy	medium	72.78%	12.00%	70.82%	68.03%	111,973.15	381.01 s
Naive	none	15.19%	2.00%	18.88%	16.47%	225.95	2.98 s
Naive	medium	48.73%	8.00%	47.42%	45.55%	4,493.37	28.87 s
CRAFT (pass@1)	none	2.53%	0.00%	4.72%	3.61%	1,135.83	7.86 s
CRAFT (pass@1)	medium	61.08%	14.00%	54.08%	54.33%	6,928.30	53.52 s
CRAFT (compose@1)	none	49.68%	8.00%	45.71%	44.95%	1,135.83	29.66 s
CRAFT (compose@1)	medium	64.24%	14.00%	63.09%	60.58%	6,928.30	66.60 s
CRAFT (compose@4)	none	48.10%	30.00%	51.72%	49.04%	1,905.10	46.40 s
CRAFT (compose@8)	none	55.38%	44.00%	56.87%	55.53%	2,929.47	69.99 s

The main text reports each method's reasoning_effort=none row. The shared model-call parameters for the non-reasoning comparison are summarized separately in Table 21.

Table 21: Sampling parameters used in the non-reasoning target-hidden tool comparison.

Method	Sampling budget / task	Temperature / top-p	Completion cap	Reasoning
AutoSpec	8	1.0 / 1.0	8,192	none
SESpec	1 initial + up to 3 refinements per phase	1.0 / 1.0	8,192	none
Clause2Inv	adaptive; 1 per call	1.0 / 1.0	8,192	none
Daikon	no model call	–	–	–
Naive	1	1.0 / 1.0	8,192	none
CRAFT	1, 5, or 2×5	1.0 / 1.0	8,192	none
Loopy	15 initial + up to 7 repairs	0.7 / 1.0	8,192	none

Token totals are means over tasks with at least one model call; accuracy always keeps the 832-task denominator.

I.8 Tool-Specific Behavior under Target Hiding

The evaluated systems interact differently with the target-hidden protocol. The following observations clarify their effective inference procedures.

AutoSpec. AutoSpec pools eight parallel proposals in its native orchestration. Its reported result therefore reflects a multi-proposal pipeline.

Clause2Inv. Clause2Inv normally constructs false-state counterexamples from the postcondition. Because the postcondition is hidden in our protocol, that source of counterexamples is unavailable. Its required transition verification conditions are also unavailable for the 466 Loopy programs; these unsupported instances remain failures in the common denominator.

Loopy. Although 98.8% of Loopy's initial responses contain textual loop-invariant annotations, its native extractor passes only 1.75% of initial responses to Houdini. This indicates an interface bottleneck between generated text and the clauses accepted by its downstream pipeline.

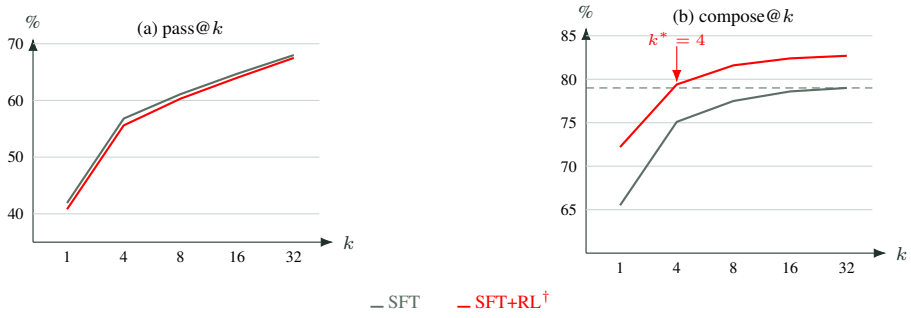

Figure 3: Response-level vs. compositional effects of RL (Qwen3-8B, SFT initialization). Red curves and values are provisional mock results. (a) pass@k remains nearly flat, consistent with redistribution rather than support expansion (Yue et al., 2025). (b) compose@k shifts upward at small budgets: the RL policy reaches its initialization's compose@32 (dotted line) at k^* samples, and compose@32 itself does not degrade.

J Detailed Limitations

Program scope. Our implementation targets numerical programs with one scalar-integer loop. This scope follows the dominant setting in prior loop-invariant benchmarks, but excludes nested loops and invariants over arrays, heaps, or recursively defined predicates. Such invariants require richer representations and proof dependencies, while available training corpora contain too few examples to support the discriminative signal used here. Our state perturbations cannot yet construct reliably informative negatives for inductive predicates or interacting nested loops (Liu et al., 2024).

Inference engine. We use vLLM for all local base and trained checkpoints because its throughput makes broad sampling practical and its consistent use avoids an additional train–test mismatch. API models retain their provider serving interfaces and chat templates under the reported decoding and reasoning controls.

Verifier and specification language. Our filtering and all final judgments use Frama-C/WP, and candidates use ACSL. Parser behavior, generated obligations, prover integration, and syntax affect which clauses survive. The quantitative conclusions may not transfer unchanged to other verifiers, specification languages, or source languages.

The ideal Houdini theorem assumes exact logical decisions. The implemented WP predicate is deliberately one-sided: only a proved obligation is accepted, while unknown results, failures, and timeouts cause deletion. Consequently, the implemented result is verifier-accepted but need not be the greatest jointly inductive subset of the candidate pool.

Reward surrogate. The reward is empirical trace coverage plus a smaller, full-group-conditioned Shapley bonus. It has three deliberate limitations. First, it computes the exact Shapley value only for the fixed coverage sets obtained after filtering the full group; it does not measure marginal contribution to a pipeline that re-filters every coalition. Second, a support-only clause can enable joint inductiveness but receives no direct reward if it rejects no negative trace. Third, coverage weights traces rather than states, so a multi-state continuation and a singleton each count once and alternative grouping of the same states could change the score. These choices make the reward tractable, but it is not a complete measure of compositional proof value.

Finite candidate-pool approximation. Before Houdini, the pipeline implementation applies a lightweight AST-based filter to parse, normalize, and deduplicate candidate clauses. This filter is conservative and does not provide a completeness guarantee for the full ACSL expression language: a semantically useful clause that is not handled by its restricted parser can be omitted. In addition, compose@k caps the candidate union passed to verification at 300 clauses to keep WP cost bounded. When an extreme response pool exceeds this limit, later clauses may be truncated even if they add useful information. Both effects can make the reported compose@k underestimate an ideal unbounded implementation. They affect candidate recall, not the validity of reported successes, because every retained result must still pass the full Frama-C/WP final judge.

Feedback regime. We do not evaluate iterative optimization from verifier feedback. Our results characterize target-independent training and finite-budget composition, not settings with rich counterexamples or target-directed repair.

K Why Thinking Is Disabled for Local Models

For the non-reasoning API experiments reported in the main paper, GPT-family requests set reasoning_effort=none, compatible endpoints disable thinking explicitly, and Claude Sonnet disables adaptive thinking with a zero thinking budget. For all locally served checkpoints, including the base probes and models trained in our pipeline, we disable chain-of-thought (CoT) during synthetic data construction, training rollouts, and evaluation. Applying the same setting at all three stages avoids a train–test distribution shift. This choice has two practical motivations.

CoT substantially increases generation cost. Explicit CoT adds reasoning tokens to every rollout, so its cost is multiplied by the sampling budget, while the required output is only a short, machine-parseable set of ACSL clauses.

Invariant clauses compose, but CoTs do not. Invariant output is decomposable: Houdini can remove a failing clause while retaining its siblings, and the survivors compose across rollouts. A CoT has no such clause-level correspondence—correct and incorrect reasoning interleave in one trace, and fragments from different rollouts cannot be merged into one faithful rationale for the composed set. CoT supervision is therefore not aligned with the clause-wise filter-and-compose target.

L REWARD COMPUTATION: POOL BEFORE FILTER

Algorithm 3 is the exact reward path used in training. It records provenance before union de-duplication, invokes the filtering pipeline once on the full group, and only then computes per-rollout credit. No positive-state or proof check is applied to an individual rollout: the clauses are combined first, and the positive-state, syntax, and Houdini checks all run on that pooled set.

Why filtering must follow pooling. Houdini is not distributive over rollout sets:

$$\mathsf{Filter}_{\mathcal{L}}\left(\bigcup_{i=1}^{G} C_i^{\mathrm{raw}}\right) \neq \bigcup_{i=1}^{G} \mathsf{Filter}_{\mathcal{L}}(C_i^{\mathrm{raw}}) \quad \text{in general.}$$

A clause that fails preservation in isolation can be inductive together with a companion clause proposed by another rollout. Filtering each rollout first would delete that clause irreversibly and would therefore score a different object from the pooled clause set used at inference. Pooling first preserves such cross-rollout support; provenance assignment after filtering then separates the shared proof context from per-rollout credit.

Algorithm 3 Pooled filtering, provenance assignment, and Shapley reward

Input: loop $\mathcal{L}$, rollout responses $Y_{1:G}$, positives $\mathcal{E}$, negatives $\mathcal{N}$, weights (w_c, w_g)
Output: rewards $r_{1:G}(\mathcal{G})$, pooled survivors $C_{\mathcal{G}}^{\mathrm{surv}}$

for $i = 1, \ldots, G$ **do** $\bar{A}_i \leftarrow \mathrm{Cap}_K(\mathrm{Normalize}(Y_i))$; $C_i^{\mathrm{raw}} \leftarrow \mathrm{set}(\bar{A}_i)$ ▷ *decompose only*
  for $c \in C_i^{\mathrm{raw}}$ **do** $\Omega(\mathrm{nkey}(c)) \leftarrow \Omega(\mathrm{nkey}(c)) \cup \{i\}$ ▷ *record owners*
$C_{\mathcal{G}}^{\mathrm{pool}} \leftarrow \mathrm{ConservativeDedup}(\bigcup_{i=1}^{G} C_i^{\mathrm{raw}})$ ▷ *pool*
$C_{\mathcal{G}}^{\mathrm{surv}} \leftarrow \mathsf{Filter}_{\mathcal{L}}(\mathsf{PF}_{\mathcal{E}}(C_{\mathcal{G}}^{\mathrm{pool}}))$ ▷ *one filter call*
for $i = 1, \ldots, G$ **do** $C_i^{\mathrm{attr}}(\mathcal{G}) \leftarrow \{c \in C_{\mathcal{G}}^{\mathrm{surv}} : i \in \Omega(\mathrm{nkey}(c))\}$ ▷ *assign survivors*
  $R_i(\mathcal{G}) \leftarrow \{\nu \in \mathcal{N} : \exists c \in C_i^{\mathrm{attr}}(\mathcal{G}),\ \nu \not\models c\}$ ▷ *rejected traces*
if $\mathcal{N} = \varnothing$ **then return pooled binary validation rewards**
for $\nu \in \mathcal{N}$ **do** $f_{\mathcal{G}}(\nu) \leftarrow \sum_{j=1}^{G} \mathbf{1}[\nu \in R_j(\mathcal{G})]$
for $i = 1, \ldots, G$ **do** $\mathrm{cov}_i(\mathcal{G}) \leftarrow |R_i(\mathcal{G})|/|\mathcal{N}|$, $\phi_i(\mathcal{G}) \leftarrow |\mathcal{N}|^{-1} \sum_{\nu \in R_i(\mathcal{G})} f_{\mathcal{G}}(\nu)^{-1}$
  $r_i(\mathcal{G}) \leftarrow w_c\, \mathrm{cov}_i(\mathcal{G}) + w_g \phi_i(\mathcal{G})$
return $(r_{1:G}(\mathcal{G}), C_{\mathcal{G}}^{\mathrm{surv}})$

The implementation realizes the owner map by matching each rollout's canonical normalization keys against the survivor keys and retains the rollout's original spelling in diagnostics. Thus spellings identified by the listed normalization rules share one pooled proof verdict while each proposer remains an owner; duplicate lines inside one rollout are represented by one entry in the set-valued owner map, while candidate de-duplication occurs only after pooling. Since every pooled survivor has at least one owner,

$$\bigcup_{i=1}^{G} R_i(\mathcal{G}) = \mathrm{rej}(C_{\mathcal{G}}^{\mathrm{surv}}),$$

which gives the efficiency equality in Equation (1). If two clauses from different rollouts are inductive only together, both can now survive. A rollout receives credit for the negatives rejected by its own attributed clause; a support-only clause that rejects none receives no direct credit, even if it enabled another survivor. Algorithm 3 computes no separate supporting-clause score.

What Shapley value is computed. Conditioned on the observed full group, $R_{1:G}(\mathcal{G})$ are fixed player sets and Algorithm 3 computes the exact Shapley value of the coverage game $v_{\mathcal{G}}$. It does *not* compute the Shapley value of the complete pool–filter–prove system or of a different game that re-runs Houdini after removing players. Even leave-one-out credit under the set-level coverage value,

$$\Delta_i = \mathrm{cov}\left(\mathsf{Filter}_{\mathcal{L}}\left(\bigcup_{j=1}^{G} C_j^{\mathrm{raw}}\right)\right) - \mathrm{cov}\left(\mathsf{Filter}_{\mathcal{L}}\left(\bigcup_{\substack{1 \le j \le G \\ j \neq i}} C_j^{\mathrm{raw}}\right)\right),$$

requires at least $G + 1$ union-level Houdini evaluations per prompt and still is not full coalition Shapley; exact coalition evaluation requires up to 2^G filtered subsets. The full-group-conditioned coverage game preserves informative per-rollout variation with one pooled filter call.

Cost check. We compared this implementation with the historical independent-filtering path on a fixed, stratified sample of 30 archived tasks (10 Linear, 10 NLA, and 10 Loopy), eight rollouts per task, using the same frozen negatives and a 5-second Frama-C/WP budget per obligation. The old path separately filtered each rollout and also filtered the union for the group score, whereas the new path reuses its single pooled result for both reward and group diagnostics.

	Independent filter	Pooled + provenance
Mean filter calls / task	8.6	1.0
Total scoring time (s)	824.39	655.60
Median task time (s)	16.82	6.48

Table 22: Runtime comparison of reward-filter order on 240 archived rollouts. Timing includes syntax, positive-state, and Frama-C/WP filtering.

Pooled filtering lowers total time by 20.5% and median task time by 61.5%, while reducing the number of authoritative filter calls to one.